


# An Autonomous Adaptive Cyber Immune System for Indian SMEs

## FINAL YEAR

## PROJECT REPORT

Submitted by

Dhakshin Kumar VS (23619111)

Keerthivahithan (23619144)

Narine (23619145)

Deepak Sanjay (23619108)

in partial fulfilment of the requirements for the award of the degree of

## BACHELOR OF TECHNOLOGY CSE - ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING




**DEPARTMENT OF CSE (AI & ML)**

**SCHOOL OF ENGINEERING**
**VELS INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)**
**CHENNAI**

**ACADEMIC YEAR 2026-2027**

## BONAFIDE CERTIFICATE

Certified that this project report entitled “AN AUTONOMOUS ADAPTIVE CYBER IMMUNE SYSTEM FOR INDIAN SMEs” is the bona fide work of Dhakshin Kumar VS (23619111), Keerthivahithan (23619144), Narine (23619145) and Deepak Sanjay (23619108), students of the Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), VELS Institute of Science, Technology & Advanced Studies (VISTAS), Chennai, submitted in partial fulfilment of the requirements for the award of the Bachelor of Technology degree during the academic year 2026-2027.

SIGNATURE OF THE HOD

**Dr. K. Ulaga Priya M.E.,(Ph.d)**
Associate Professor & Head
Department of CSE (AI & ML)
School of Engineering, VISTAS
Chennai

SIGNATURE OF THE SUPERVISOR

**Mrs. S. J. Kavitha M.E.,(Ph.d)**
Assistant Professor
Department of CSE (AI & ML)
School of Engineering, VISTAS
Chennai

Submitted for the Project Work and Viva-Voce examination held on

_____

at VELS Institute of Science, Technology & Advanced Studies (VISTAS), Chennai - 600 117.

**INTERNAL EXAMINER**

**EXTERNAL EXAMINER**

## ACKNOWLEDGEMENT

It is a pleasure to acknowledge the guidance and support received during this final-year project. We thank VELS Institute of Science, Technology & Advanced Studies (VISTAS), the School of Engineering and the Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning) for providing the academic environment and resources in which this work was carried out.

We express our sincere gratitude to our project guide, Mrs. S. J. Kavitha, for her direction, constructive feedback and encouragement throughout the project. We also thank Dr. K. Ulaga Priya, Head of the Department, for her support and for enabling us to complete this work. We acknowledge the maintainers of the open-source systems, libraries and technical documentation that informed our study and implementation.

Finally, we thank our families, friends and peers for their patience and encouragement during the completion of this project report.

| | |
|---|---|
| **Dhakshin Kumar VS** | **23619111** |
| **Keerthivahithan** | **23619144** |
| **Narine** | **23619145** |
| **Deepak Sanjay** | **23619108** |

# TABLE OF CONTENTS

**ABSTRACT** ..... **6**
**LIST OF FIGURES** ..... **7**
**LIST OF TABLES** ..... **8**
**LIST OF ABBREVIATIONS** ..... **9**
**INTRODUCTION** ..... **10**
1.1 Context and motivation ..... 10
1.2 Problem statement ..... 10
1.3 Aim and objectives ..... 11
1.4 Scope and assumptions ..... 11
1.5 Contributions ..... 12
1.6 Report structure ..... 12
**LITERATURE REVIEW AND FOUNDATIONS** ..... **13**
2.1 Cybersecurity risk management for smaller organisations ..... 13
2.2 Host telemetry and eBPF ..... 13
2.3 Rule-based and behavioural detection ..... 14
2.4 Adaptive thresholds and response governance ..... 14
2.5 Streaming, storage and observability ..... 15
2.6 Deception and adversary mapping ..... 15
2.7 Research gap and project position ..... 16
**REQUIREMENTS AND FEASIBILITY** ..... **17**
3.1 Stakeholders and operating context ..... 17
3.2 Functional requirements ..... 17
3.3 Quality, security and privacy requirements ..... 18
3.4 Feasibility, assumptions and constraints ..... 18
**SYSTEM DESIGN AND METHODOLOGY** ..... **20**
4.1 Design principles ..... 20
4.2 Logical architecture ..... 20
4.3 Event processing and data flow ..... 21
4.4 Adaptive detection and response governance ..... 22
4.5 Data model and trust boundaries ..... 23
**IMPLEMENTATION AND STATUS** ..... **24**
5.1 Runtime stack and deployment ..... 24
5.2 Linux telemetry and event bridge ..... 24
5.3 Ingestion, queues and persistence ..... 25
5.4 Detection and enrichment ..... 25
5.5 Adaptation, explanation and response ..... 26
5.6 Deception, API and dashboard ..... 26

5.7 Implementation status ..... 27
**TESTING AND RESULTS ..... 28**
6.1 Verification method ..... 28
6.2 Test artefacts in the repository ..... 28
6.3 Results supported by repository evidence ..... 29
6.4 Dashboard output screenshots ..... 30
6.5 Acceptance criteria and measurement gaps ..... 31
**SECURITY, PRIVACY AND LIMITATIONS ..... 32**
7.1 Security and privacy analysis ..... 32
7.2 Limitations and open risks ..... 33
7.3 Ethical and operational considerations ..... 33
**CONCLUSION AND FUTURE WORK ..... 35**
8.1 Conclusion ..... 35
8.2 Prioritised future work ..... 35
**APPENDIX A: REPOSITORY EVIDENCE AND COMPONENT REVIEW ..... 37**
A.1 Test inventory ..... 37
A.2 Repository snapshot ..... 38
A.3 Component-by-component source review ..... 38
**APPENDIX B: SELECTED SOURCE LISTINGS ..... 45**
**APPENDIX C: REQUIREMENTS AND VERIFICATION PLAN ..... 53**
C.1 Requirements catalogue ..... 53
C.2 Selected verification protocols ..... 59
C.3 SME pilot and evaluation plan ..... 67
C.4 Evidence-status key ..... 68
**REFERENCES ..... 69**

## ABSTRACT

Small and medium-sized enterprises need security monitoring that can collect useful host signals, relate them to suspicious behaviour and present evidence in an operationally understandable form. This project studies GhostLayer, a repository-backed cyber-monitoring prototype intended for Indian SMEs. Its design brings endpoint telemetry, event processing, rule and behavioural detection, adaptive analysis and operator-facing services into one software project. The report evaluates the current repository rather than treating the project title as evidence that every capability is complete. The reviewed source and configuration define a Linux endpoint path built around eBPF collection, a Python event bridge, TCP ingestion, DuckDB and Parquet storage, Redis Streams, detection modules, adaptive thresholds, a dashboard API and a React interface. Compose configuration also describes canary, causal-analysis, Prometheus and Grafana services. Source files include response-governance logic and a separate Windows monitoring path, but evidence for runtime integration and several component statuses is incomplete or conflicting. The report reviews the repository at commit 4dfb877 and traces implementation claims to source files, configuration and test artefacts. Test scripts cover detector behaviour, integration, API regressions, performance and false-positive checks, but they were not executed for this report. No independently reproducible detection-accuracy, latency, throughput or endpoint-overhead result is asserted. Two dashboard screenshots supplied by the project team illustrate the reported interface; their displayed values were not independently reproduced. The assessment concludes that GhostLayer provides a substantial prototype foundation, while operational validation, secure deployment defaults, data-quality evaluation, customer isolation and documented detection gaps remain unresolved before SME production use.



**Keywords:** cybersecurity, eBPF, endpoint telemetry, anomaly detection, adaptive thresholds, Indian SMEs, incident response

## LIST OF FIGURES


| Figure No. | Figure Name | Page No. |
|---|---|---|
| Figure 4.1 | GhostLayer logical architecture | 20 |
| Figure 4.2 | Event processing and learning loop | 21 |
| Figure 4.3 | Response governance | 22 |
| Figure 6.1 | Dashboard overview output supplied by the project team | 30 |
| Figure 6.2 | Alert-detail output supplied by the project team | 30 |

## LIST OF TABLES


| Table No. | Table Name | Page No. |
|---|---|---|
| Table 2.1 | Technical foundations and their role in this project | 16 |
| Table 3.1 | Functional requirements and repository evidence | 17 |
| Table 5.1 | Repository implementation status | 27 |
| Table 6.1 | Test artefacts and evidence status | 28 |
| Table 7.1 | Security and operational risks | 33 |
| Table A.1 | Claim-to-source traceability | 37 |
| Table C.1 | Proposed staged pilot timeline | 67 |
| Table C.2 | Measures and reporting boundaries | 67 |
| Table C.3 | Evidence-status key | 68 |

## LIST OF ABBREVIATIONS

| Abbreviation | Full Form |
|---|---|
| AI | Artificial intelligence |
| API | Application programming interface |
| ATT&CK | Adversarial Tactics, Techniques, and Common Knowledge |
| BPF | Berkeley Packet Filter |
| C2 | Command and control |
| CSE | Computer Science and Engineering |
| EDR | Endpoint detection and response |
| eBPF | Extended Berkeley Packet Filter |
| ETW | Event Tracing for Windows |
| FP | False positive |
| HOD | Head of Department |
| IDS | Intrusion detection system |
| JSON | JavaScript Object Notation |
| LSM | Linux Security Module |
| ML | Machine learning |
| MSP | Managed service provider |
| PII | Personally identifiable information |
| SME | Small and medium-sized enterprise |
| SQL | Structured Query Language |
| VISTAS | VELS Institute of Science, Technology & Advanced Studies |

# CHAPTER 1
# INTRODUCTION

## 1.1 Context and motivation

Cybersecurity monitoring is an operational capability: it depends on collecting relevant signals, interpreting them in context, communicating risk and supporting a proportionate response. The NIST Cybersecurity Framework 2.0 provides a risk-management vocabulary intended for organisations of different sizes and sectors; it does not prescribe one technical product or guarantee a particular outcome [2].

GhostLayer applies that general problem to Indian SMEs. Its repository describes a digital immune-system concept built around endpoint telemetry, detection, incident context and operator-facing services. The value of this project is therefore assessed by the traceable engineering surface in the repository, rather than by claims about market-wide breach rates, customer savings or detection accuracy for which the reviewed repository contains no independently reproducible evidence.

The name GhostLayer is used for the academic project. The source repository also uses the labels Ghost IT and Ghost Layer Technologies. This report keeps those source labels when identifying code and configuration, while using the confirmed project title on the cover.

## 1.2 Problem statement

A small organisation may have limited time to correlate host events, distinguish unusual behaviour from routine activity and maintain a continuous monitoring stack. A useful prototype should make the event path understandable, allow detections to be inspected, and expose its limitations clearly. The engineering problem addressed here is to build a modular host-monitoring and alerting path that can collect selected activity, transform and persist events, evaluate detections, and present evidence through a dashboard.

The repository is not evidence of a fully deployed managed security service. It is a software prototype with a Docker Compose definition, host-agent source, detection modules and test scripts. The operational problem is consequently framed as a design-and-implementation problem, with deployment and efficacy validation identified as required next steps.

### 1.3 Aim and objectives

Aim: to design and document an adaptive cyber-monitoring prototype for Indian SMEs, and to evaluate the current implementation honestly against its source code, configuration and test artefacts.

1. Implement a Linux telemetry path using eBPF hooks and a userspace event bridge.
2. Ingest, normalize, queue and persist event records for querying and alert review.
3. Combine rule-based and behavioural detection with context and adaptive thresholds where the code path supports them.
4. Provide API and dashboard surfaces for events, alerts, incidents and guidance.
5. Add deception and response-governance components as modular parts of the wider architecture.
6. Assess the current repository state, identify test evidence and document the remaining risks and work.

### 1.4 Scope and assumptions

The assessed scope is the public GitHub repository at commit 4dfb877, retrieved on 22 October 2026 [1]. The main deployment path is Linux-based. A Windows monitoring directory also exists, but its completeness and runtime integration are not treated as verified. The project covers a prototype event pipeline and security-monitoring workflow; it does not establish a certified product, a production service-level objective, a tested Indian regulatory compliance claim, or a representative customer evaluation.

The system assumes that an administrator can install and run a privileged endpoint agent, provide the deployment configuration and protect event data. It

also assumes that operators review alerts and that automated actions remain bounded by explicit safety logic. These assumptions must be tested with intended users before operational deployment.

### 1.5 Contributions

The report contributes a traceable system description, a component-level architecture, a functional requirement set, and an implementation-status matrix. It also connects the repository to established technical foundations in eBPF, anomaly detection, risk management and adversary-behaviour mapping [2]-[7]. The status matrix is deliberately conservative: a source file is evidence that code exists, not evidence that a service has been deployed or that a detection has achieved a measured quality target.

### 1.6 Report structure

Chapter 2 introduces the relevant technical foundations and positions the work. Chapter 3 states requirements and feasibility assumptions. Chapter 4 explains the architecture, event flow and response-governance design. Chapter 5 maps those designs to repository components and classifies their status. Chapter 6 inventories verification artefacts and the results that can be supported. Chapter 7 addresses security, privacy, limitations and ethical operation. Chapter 8 concludes and prioritises future work.

# CHAPTER 2
# LITERATURE REVIEW AND FOUNDATIONS

## 2.1 Cybersecurity risk management for smaller organisations

The NIST Cybersecurity Framework (CSF) 2.0 organises cybersecurity outcomes around Govern, Identify, Protect, Detect, Respond and Recover. It is a risk-management framework rather than a prescribed implementation blueprint, and it is intended to support organisations with different levels of maturity [2]. GhostLayer is best interpreted as a technical contribution to selected Detect and Respond activities, with supporting operational visibility. The repository does not demonstrate coverage of every CSF outcome and should not be described as CSF-compliant.

For an SME-oriented system, design choices need to consider deployment effort, operator understanding, safe defaults and the consequences of false alarms. These are design requirements to evaluate, not benefits already proven by the repository. A customer study would be required to measure installability, workload reduction, alert usefulness and cost.

## 2.2 Host telemetry and eBPF

eBPF is a Linux kernel mechanism for attaching verified programs to supported hooks for tracing, networking and security-related instrumentation [3]. It is relevant to this project because host events can be collected close to kernel activity without requiring the repository to replace or patch the Linux kernel. The GhostLayer eBPF source contains tracepoint, kprobe and LSM-style event handlers, while a userspace loader and Python bridge provide a path for event serialization and forwarding [1].

Kernel telemetry is constrained by kernel versions, available BTF information, privileges, verifier rules and the behaviour of host workloads. An event collector also requires careful resource controls and failure handling. The repository Makefile documents generation of vmlinux.h from kernel BTF, compilation of a

BPF object, and building a userspace loader; this is build configuration evidence, not proof of successful compilation on a range of SME endpoints [1].

### 2.3 Rule-based and behavioural detection

Rule-based detection maps observable events to explicit conditions and can produce interpretable reasons. Behavioural detection instead compares feature windows with a learned or maintained baseline. GhostLayer includes multiple detector modules and a behavioural engine with event windows, per-entity feature vectors and EWMA-style baseline support. Its Isolation Forest component can train or load a model and score feature vectors; Isolation Forest was originally presented as an efficient tree-based method for identifying anomalies [4].

An anomaly score is not a verdict that an event is malicious. Models may flag legitimate rare behaviour, fail to detect a novel attack, or behave differently as the workload changes. A robust evaluation therefore needs a representative labelled or adjudicated dataset, per-class precision and recall, false-positive analysis, drift monitoring, and thresholds chosen for the operational cost of misses and alerts. No such complete evaluation is reported by the reviewed commit.

### 2.4 Adaptive thresholds and response governance

The repository includes a persistent adaptive-threshold component that records scores and computes locally calibrated percentile thresholds within configured bounds. It also contains a safety-governor function that can adjust a proposed response tier based on assessed criticality, blast radius and false-positive history, including a human-confirmation outcome for selected conditions [1]. This is a plausible software pattern for making a decision path more context-aware.

Threshold adaptation requires safeguards against feedback contamination: if malicious observations enter the normal baseline, the system can learn to tolerate the activity it should detect. This report therefore treats adaptation as implemented code with unverified field behaviour. The required validation

includes controlled clean and attack traffic, poisoned-baseline tests, reset and rollback procedures, and explicit human oversight rules.

### 2.5 Streaming, storage and observability

The deployment uses DuckDB for analytical storage and defines a hot-buffer and Parquet-oriented pipeline. DuckDB documents an embedded analytical SQL engine that can be used in local and server settings [5]. Redis Streams offer append-only event records and consumer groups; the detection engine uses stream operations in the repository [6]. Together these components express a design for ingesting, buffering and consuming event records.

The repository also defines Prometheus and Grafana services. Their presence in a Compose file does not establish useful dashboards, alert thresholds or service-level observability. The technical-debt register specifically marks panels for mean time to detect and false-positive rate, and migration-trigger alerts wired into Grafana, as not started [1].

### 2.6 Deception and adversary mapping

Canary tokens, monitored decoy files and honeypot services can create high-signal events when accessed. GhostLayer contains canary token and server modules as well as a honeypot orchestrator and network-isolation helpers. The repository evidence ranges from files labelled complete to a honeypot README marked 0 percent and an orchestrator labelled 40 percent. The safe conclusion is that deception components are present, while full isolation and deployment maturity are unverified [1].

MITRE ATT&CK is a knowledge base of adversary tactics and techniques grounded in observed behaviour [7]. It can support coverage mapping and simulation design. A mapping or simulation result is not evidence that a production system detects all instances of a technique. The technical-debt register names gaps such as brute-force tracking and ingress-tool-transfer context, which should remain visible in any coverage claim [1].

### 2.7 Research gap and project position

The project combines kernel-level event collection, a local analytics path, rule and behavioural modules, adaptive decision code and operator-facing services in a single repository. This integration is its engineering focus. The current gap is not the absence of all component code; it is the need to establish a reproducible end-to-end deployment, validated datasets, measured detection quality, safe defaults and customer-centred usability.

**Table 2.1 Technical foundations and their role in this project**

| Foundation | What it provides | Project use and boundary |
|---|---|---|
| NIST CSF 2.0 [2] | Risk-management outcomes and common vocabulary. | Used to frame selected monitoring activities; no claim of full framework coverage. |
| Linux eBPF [3] | Verified kernel instrumentation through supported hooks. | Used by the Linux collector source; endpoint compatibility is not measured. |
| Isolation Forest [4] | Anomaly scoring based on isolation paths. | Model code exists; accuracy and generalisation are not established. |
| DuckDB and Redis Streams [5], [6] | Analytical storage and event-stream primitives. | Used in the pipeline design; scale and durability targets are unverified. |
| MITRE ATT&CK [7] | A vocabulary for observed adversary tactics and techniques. | Supports mapping and simulations; it is not a coverage guarantee. |

# CHAPTER 3
# REQUIREMENTS AND FEASIBILITY

## 3.1 Stakeholders and operating context

The primary operational user is an SME administrator or security operator who needs a small number of understandable alerts, a way to inspect associated host events and a clear indication of recommended action. A managed service provider (MSP) operator may oversee multiple customer views. A system administrator installs the Linux agent and manages configuration, permissions and updates. Project evaluators need evidence of functional behaviour and measured quality rather than source-code volume alone.

The deployment context assumes a supported Linux host, administrative access for the kernel agent, network connectivity between the agent and pipeline, persistent storage, and an operator who can investigate alerts. The source tree contains code paths for Windows monitoring, but that path is outside the verified baseline for this report.

## 3.2 Functional requirements

**Table 3.1 Functional requirements and repository evidence**

| ID | Requirement | Evidence in the repository |
|---|---|---|
| FR-01 | Collect selected endpoint and kernel events. | eBPF handlers and userspace loader in agent/ebpf/ [1]. |
| FR-02 | Serialize and forward event batches to the pipeline. | Python bridge and TCP event ingestion in pipeline/server_v2.py [1]. |
| FR-03 | Persist and query events and alerts. | DuckDB / Parquet pipeline and HTTP query routes [1]. |
| FR-04 | Score activity using rules and behavioural features. | Detection engine, detector modules and behavioural model code [1]. |
| FR-05 | Provide incident context and guarded decision support. | Causal modules, response governor and dashboard API routes [1]. |
| FR-06 | Let authorised users review alerts and system summaries. | Dashboard API and React application source [1]. |
| FR-07 | Generate deception-related security signals. | Canary server, token and watcher modules; deployment maturity is unverified [1]. |
| FR-08 | Expose service health and selected operational metrics. | Compose health checks and Prometheus/Grafana configuration [1]. |

### 3.3 Quality, security and privacy requirements

- Performance: collect and query events without unacceptable impact on SME endpoints. The acceptance script defines resource checks, but no independent values are reported.
- Reliability: buffer short service interruptions, preserve event ordering where needed, and expose health state. The repository contains retry, queue and health-check code; recovery guarantees remain unmeasured.
- Security: minimise privileged execution, authenticate dashboard access, protect service secrets, secure agent-to-server transport, and keep tenant boundaries explicit.
- Privacy: disclose the telemetry collected, minimise command-line and path retention, set retention periods, restrict operator access, and support verified data export or erasure where applicable.
- Usability: show why an alert fired, its confidence and the evidence needed for review; provide reversible or human-reviewed response for high-impact actions.
- Maintainability: keep components deployable and testable, pin dependencies and runtime versions, document the supported host matrix, and maintain reproducible CI checks.

### 3.4 Feasibility, assumptions and constraints

Technical feasibility is supported by concrete code and container definitions for a multi-service prototype. Operational feasibility is conditional: the Linux agent needs root-level access and is started manually according to start.sh, while Compose uses deployment-specific paths and settings. Economic feasibility has not been quantified; the repository includes no SME cost study, managed-service staffing model or total-cost estimate.

The project is constrained by kernel and distribution compatibility, representative training and evaluation data, the completeness of detector inputs, secret and certificate management, tenant identity, resource use, and time available for field validation. The codebase has several adjacent implementations and documentation inconsistencies. The Python version stated in the root README differs from the Python version used by the container Dockerfiles; deployment should therefore follow pinned container definitions and reconcile the documentation [1].

Feasibility conclusion: the repository supports a demonstrable academic prototype and a structured validation plan. Production deployment to customer systems requires security review, safe configuration, operating-system compatibility tests and an agreed pilot protocol.

# CHAPTER 4
# SYSTEM DESIGN AND METHODOLOGY

### 4.1 Design principles

- Collect specific signals at the endpoint, then send structured records to modular backend services.
- Separate event collection, ingestion, detection, persistence and user presentation so that each can be inspected.
- Combine explicit rules with behavioural scoring and event context; preserve reasons and evidence for analyst review.
- Treat adaptation as bounded calibration, with a safety governor between proposed and consequential actions.
- Keep privileged host collection separate from ordinary application services and expose limitations in the interface and documentation.

### 4.2 Logical architecture

Figure 4.1 summarises the source-defined services. Linux kernel telemetry is converted into structured events by the userspace path, sent to the pipeline, then written to storage and made available to the detection engine. Detection output and incident context are exposed through API and dashboard components. Deception and observability services appear as related modules. The figure is a logical view of repository wiring; it is not a deployment health report.

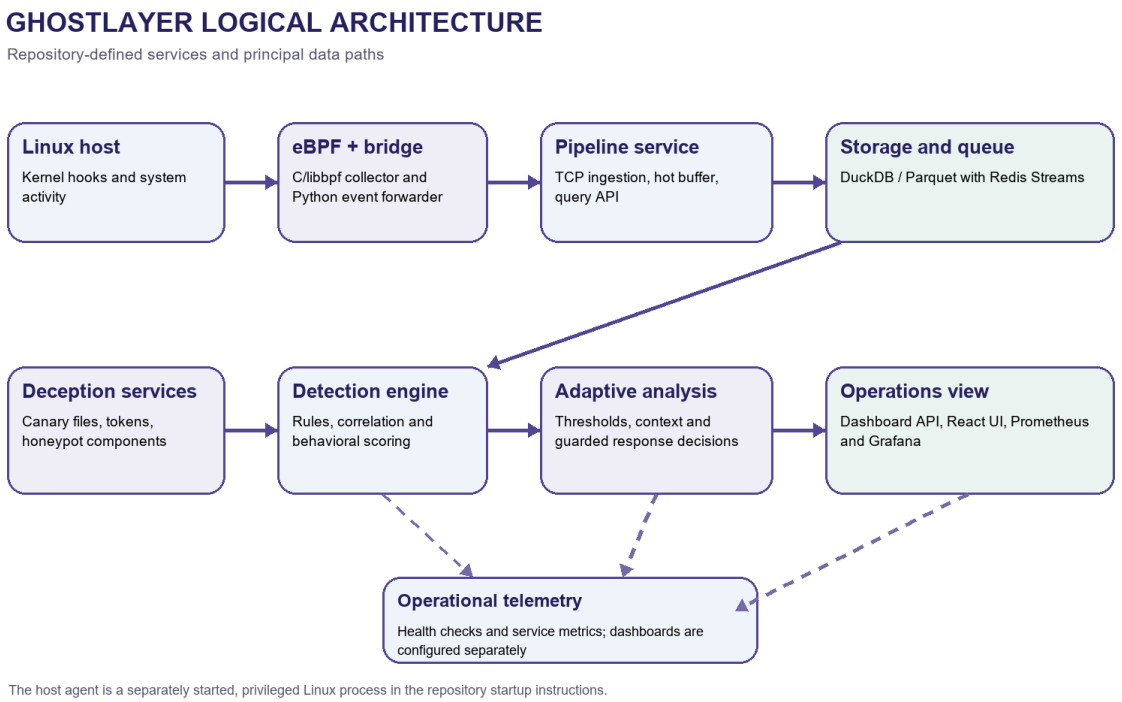



**Figure 4.1 GhostLayer logical architecture based on the reviewed repository.**

### 4.3 Event processing and data flow

The Linux eBPF program attaches to selected kernel tracepoints, probes or security hooks and serializes fields such as time, process identity, event type, path and network metadata. The userspace bridge reads events, scores or enriches selected fields and sends JSON batches to the TCP ingestion listener. The deployed pipeline image runs server_v2.py, which defines an in-memory hot buffer, Redis stream operations, a DuckDB/Parquet-oriented persistence path, health routes and query endpoints [1].

The detection service consumes event records, applies detector logic and forwards detections to the pipeline. Additional code can contribute state such as adaptive thresholds, temporal memory or causal context. API routes provide data to the dashboard. Figure 4.2 shows this lifecycle at a conceptual level; actual timing, loss behaviour and back-pressure under load must be established by measurement.

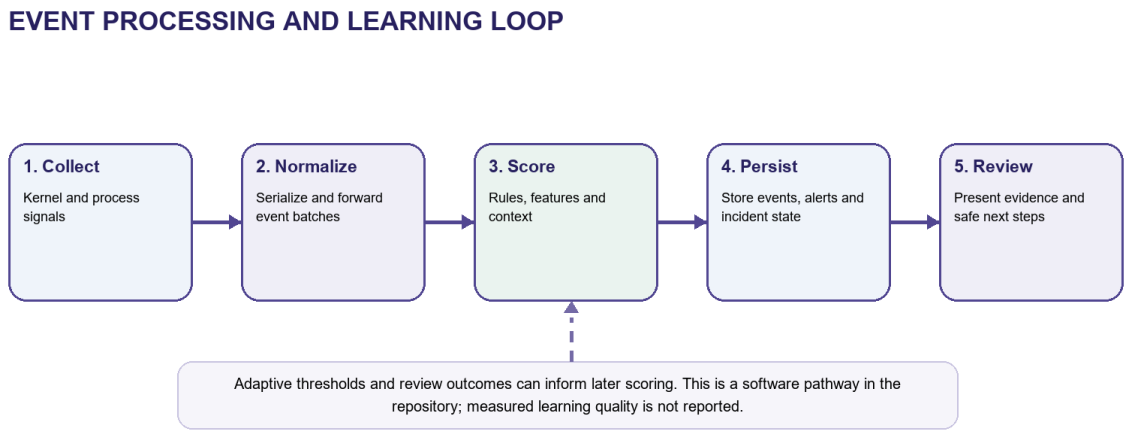



**Figure 4.2 Event processing and learning loop.**

### 4.4 Adaptive detection and response governance

The behavioural engine forms feature windows from event observations and compares scores with a baseline. The adaptive-threshold class stores observations by detection pillar and derives a bounded local threshold. A response proposal can then be evaluated using criticality, blast radius and false-positive history. The safety governor may approve, downgrade or require human confirmation. This decision path is separated conceptually from the endpoint event collection.

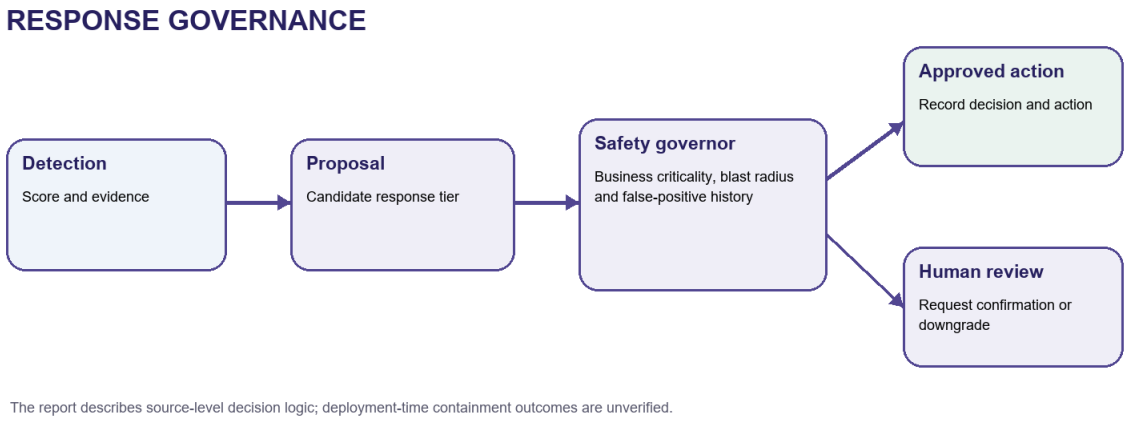



**Figure 4.3 Response governance represented by the repository decision modules.**

This architecture does not imply that every action is connected to a customer endpoint, nor that an action has been tested safely in an SME environment. High-impact actions should remain disabled until an operator-reviewed policy, rollback procedure and audit trail have been validated.

### 4.5 Data model and trust boundaries

The event schema carries process and event attributes used for filtering, scoring and incident interpretation. Queries in pipeline code reference identifiers, timestamps, process and parent-process information, user identifiers, command or process names, event types, scores, reasons, file paths and destination address or port. These fields can contain sensitive personal or business information. They should be governed as security telemetry, with access controls and retention rules.

| Boundary | Data crossing it | Control and remaining question |
|---|---|---|
| Host to collector | Kernel event records and process metadata. | Privileged eBPF path; define minimum privilege and supported kernels. |
| Collector to ingestion | JSON batches over the repository TCP path. | No TLS is visible in the bridge path reviewed; secure transport must be added or verified. |
| Pipeline to detection | Event stream and detection state. | Redis Streams consumer logic exists; recovery and duplication behaviour require tests. |
| API to dashboard user | Alerts, incidents, process summaries and actions. | Session and customer filtering code exists; authentication and tenant isolation need review. |
| Operator to response path | Proposed action and confirmation state. | Governor logic exists; execution integrations and rollback must be validated. |

# CHAPTER 5
# IMPLEMENTATION AND STATUS

## 5.1 Runtime stack and deployment

The root README describes an eBPF/libbpf kernel collector, Python backend, DuckDB, Node.js frontend and Docker. The Compose definition includes pipeline, Redis, canary, detection, causal-engine, dashboard API, dashboard, Prometheus and Grafana services. The pipeline and application Dockerfiles pin Python 3.11 while the frontend image uses Node 18. The root README's Python 3.14 statement does not match those Dockerfile bases; this is recorded as a documentation discrepancy rather than silently resolved [1].

The start script checks for Docker and starts the Compose stack. It separately instructs the operator to build and launch the eBPF agent with elevated privileges. The pipeline service also receives SYS_PTRACE capability in Compose, which should be reviewed and reduced to the minimum required before external deployment. Environment files, host paths, public dashboard API settings and monitoring credentials are deployment concerns that must be moved into secure, environment-specific configuration.

## 5.2 Linux telemetry and event bridge

The eBPF source contains handlers for process, file, authentication, network and other kernel activity, including selected DNS and TLS-related capture logic. A C userspace loader consumes BPF events, while the Python bridge parses event records, maintains a connection and transmits newline-delimited JSON batches. The build Makefile describes a BTF-based CO-RE build. These files establish an implemented source path; compilation and collection across a declared Linux support matrix are not evidenced by the report preparation.

The README and startup script present Linux as the active deployment route. A Windows-agent directory contains ETW, process-hook, serializer and forwarding source files, but its README retains an initial 0 percent status while individual

files carry completion comments. This conflicting evidence is treated as partial and unverified. The report does not claim parity between Linux and Windows.

### 5.3 Ingestion, queues and persistence

The pipeline container runs pipeline/server_v2.py. Its source documents a TCP write path into a bounded in-memory hot buffer, background flushing to Parquet files and DuckDB-backed querying. Redis Streams are used for coordination with the detection engine. The source also defines health, event, alert and process-summary routes. The repository provides persistence code, but it contains no reproduced benchmark proving throughput, retention behaviour, durability after abrupt termination or large-volume query latency.

### 5.4 Detection and enrichment

The detection engine connects to Redis and registers a consumer group, reads new records, invokes configured detector checks and sends resulting detections to the pipeline. Detector modules cover selected behaviours such as identity anomalies, suspicious command patterns, exfiltration, DNS-related activity, ransomware behaviour and other security signals. Individual detector outputs are converted into a shared detection representation at engine integration points. Some detector ideas require telemetry fields or sources that are not available in the current event schema.

Behavioural code includes feature-window construction, baseline state and Isolation Forest scoring. The model pipeline contains training, export and update modules, and the repository stores model metadata. Presence of a model artefact or training workflow does not establish the provenance of a production training corpus or the model's measured effectiveness on Indian SME workloads.

### 5.5 Adaptation, explanation and response

AdaptiveThresholds stores observed scores and returns a calibrated threshold with a fallback value when evidence is sparse. The causal-engine folder includes graph, ensemble, risk, context and reporting modules. AutonomousResponseEngine defines tiered decisions and rate-limit behaviour;

safety_governor.py can request human confirmation. Pipeline routes expose several analysis and decision functions. However, causal-engine/README.md still labels the area as 0 percent and folder-created. This mismatch means source presence and route definitions cannot be treated as proof that the whole causal service is complete or operational.

The report therefore classifies bounded threshold code and decision logic as present in source, while classifying end-to-end adaptive response as partial and unverified. The term autonomous in the project title names the intended design direction; it is not a claim that autonomous containment has passed operational safety testing.

**5.6 Deception, API and dashboard**

Canary token, token validation, file watching and an HTTP canary server are present. Honeypot code includes network-isolation helpers and a runtime orchestrator with separate deployment paths. The orchestrator comments identify a gVisor path and a Firecracker path, while the component README and inline status comments disagree about completeness. Deployment and isolation must be tested before these services are exposed.

The dashboard API has login, session verification, alert, incident, endpoint, playbook and compliance-related routes. A tenancy module maps users to customer identifiers and filters events by configured address ranges; the API includes MSP-oriented endpoints. The React frontend contains sign-in, event streaming and operational panels. These source files show user-interface functionality, not a completed security review. The frontend test file still checks the default Create React App learning link and therefore does not verify GhostLayer workflows.

Prometheus and Grafana configuration is included in the deployment. The repository's own technical-debt register marks selected Grafana panels and

migration alerts as not started. The deployment defaults and service wiring should be reviewed for secrets, public endpoint configuration and least privilege.

### 5.7 Implementation status

The classifications below refer to code and configuration in commit 4dfb877. Implemented means the source-level functionality is present and connected at the code or Compose level. Partial means only a subset or component path is present. Planned refers to explicit deferred work in the repository. Unverified means source exists but no sufficient integration or runtime evidence is present.

**Table 5.1 Repository implementation status**

| Capability | Status | Evidence-based assessment |
|---|---|---|
| Linux eBPF collection and Python bridge | Implemented in source; runtime unverified | C/BPF handlers, Makefile, loader and bridge exist; supported-kernel builds and field telemetry are not independently evidenced. |
| Pipeline, Redis, DuckDB/Parquet and API | Implemented in source/config | Compose selects server_v2.py; service routes and queue/storage code are present. Capacity and recovery are unmeasured. |
| Rule-based detection and alert forwarding | Implemented in source | Detection engine and detector modules are wired to stream consumption and pipeline forwarding. |
| Behavioural ML and adaptive thresholds | Partial; quality unverified | Feature windows, Isolation Forest code and threshold calibration exist; dataset quality and model metrics are absent. |
| Causal reasoning and autonomous response | Partial; integration unverified | Modules and routes exist, but component README says 0 percent and execution results are absent. |
| Canary and honeypot deployment | Partial; isolation unverified | Canary source and orchestrator exist; honeypot README/status evidence conflicts. |
| Dashboard and API | Implemented in source; security unverified | React and FastAPI surfaces exist; frontend test remains a starter test and auth/tenant controls need review. |
| Windows endpoint monitoring | Partial; unverified | ETW and process-monitoring source exists, but repository status is inconsistent and no parity test output is available. |
| JA4, email phishing, T1105 and T1110 coverage | Planned / explicit gap | Technical-debt register identifies missing telemetry inputs or stateful cross-event logic. |
| HA, per-agent mTLS, secrets manager, mature control plane | Deferred / not implemented | Technical-debt register states these items were rejected as premature for the current project scale. |

# CHAPTER 6
# TESTING AND RESULTS

## 6.1 Verification method

Verification for this report consisted of inspecting the repository at commit 4dfb877, reading relevant source files, Dockerfiles, Docker Compose configuration, workflow definitions, test scripts and the technical-debt register [1]. The report did not execute the test suite or start the deployment stack. Several scripts require a live Linux agent, a running pipeline or customer-like traffic; some can modify host state. A test name, assertion or printed target is not a test result.

The repository exposes a useful set of test intentions. The verification status below is intentionally limited to what can be established from checked-in files.

## 6.2 Test artefacts in the repository

**Table 6.1 Test artefacts and evidence status**

| Test artefact | Stated coverage | Evidence status |
|---|---|---|
| tests/detector_tests.py | Identity, memory-exploit, exfiltration, LOLBin and reporting components. | Script present; not executed for this report. |
| tests/e2e_integration_test.py | Cortex fusion, response decision and incident replay. | Live integration checks defined; no run output reviewed. |
| tests/regression_suite.py | Health, stats, metrics, alert history, replay and SQL-injection handling. | Uses a running API; no run output reviewed. |
| tests/v0_acceptance_test.py | Fourteen checks including signatures, kernel errors, inference latency, ransomware, API, data flow, heartbeat, resource use and gRPC. | Script requires a particular Linux environment and local services; no run output reviewed. |
| tests/perf_benchmark.py and tests/false_positive_benchmark.py | Resource targets and benign-command alert checks. | Benchmark procedures present; no current measurements reproduced. |
| tests/mitre_simulation.py, chaos_test.py and tight_poll_test.py | Selected adversary simulations, event persistence and short-window detection. | Operational scripts present; not run because they perform host actions or require live services. |
| dashboard/frontend/src/App.test.js | Default React starter text assertion. | Does not exercise GhostLayer dashboard behaviour. |
| .github/workflows | Weekly model pipeline and agent release jobs. | No workflow running the listed test suite was found in the reviewed commit tree. |

### 6.3 Results supported by repository evidence

The reviewed commit contains a multi-service Compose definition; a Linux eBPF and userspace bridge source path; a pipeline container that launches server_v2.py; a Redis-backed detection consumer; endpoint and dashboard APIs; model, adaptive-threshold and response-governor modules; and scripts covering unit-style, integration, regression and benchmark intentions. These findings establish implementation breadth at source level.

The repository's technical-debt register explicitly distinguishes completed and deferred items. It identifies missing or incomplete collection for JA4 fingerprinting, email phishing input, ingress-tool-transfer URL evidence and brute-force state tracking. It also records that several monitoring panels are not started and that broader event-loop sandboxing remains unfinished [1].

No verified confusion matrix, precision, recall, false-positive rate, detection latency, throughput, CPU overhead, memory use, recovery time or customer pilot result is reported here. The acceptance and benchmark scripts contain test procedures and thresholds, but they do not establish a result without a captured run, environment description and reproducible data.

## 6.4 Dashboard output screenshots

The following interface images were supplied by the project team. They illustrate the overview dashboard and an alert - details view visible in the captured session. Counters and alert details are screenshot values only; this repository review did not reproduce their event stream or independently verify the deployment that produce them

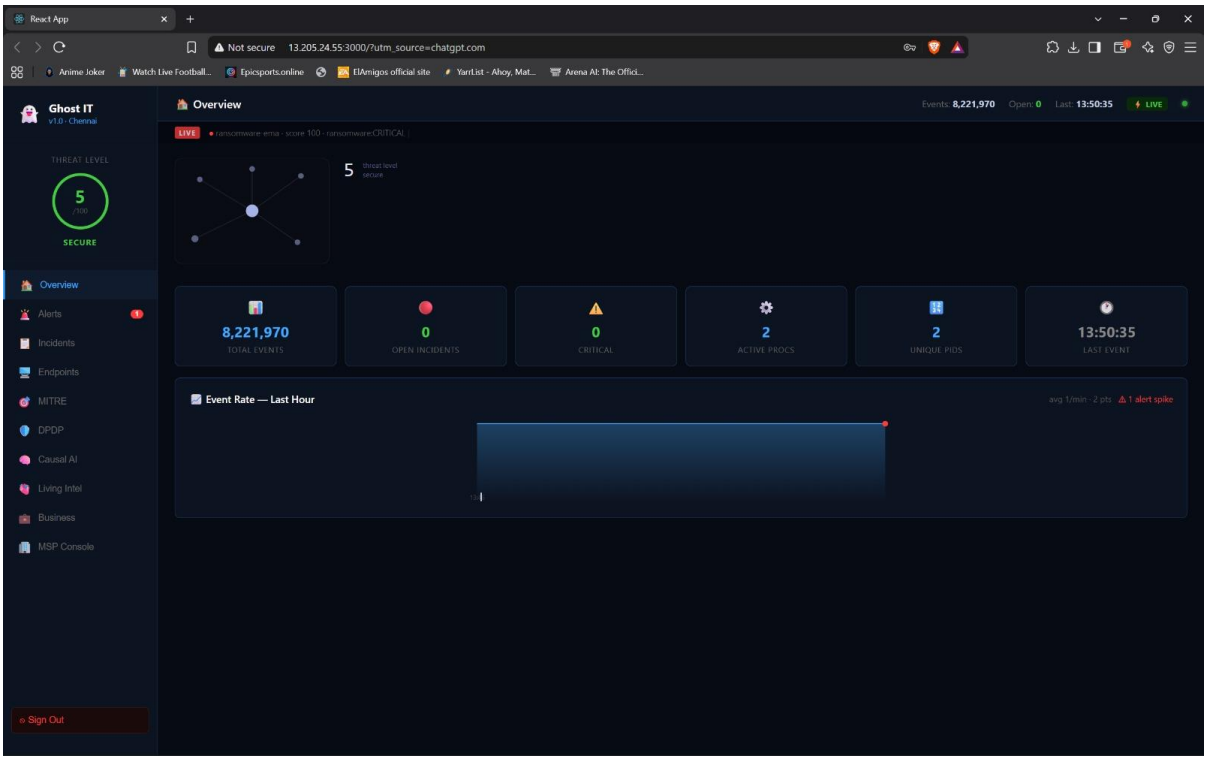



**Figure 6.1 Dashboard overview output supplied by the project team (23 October 2026).**

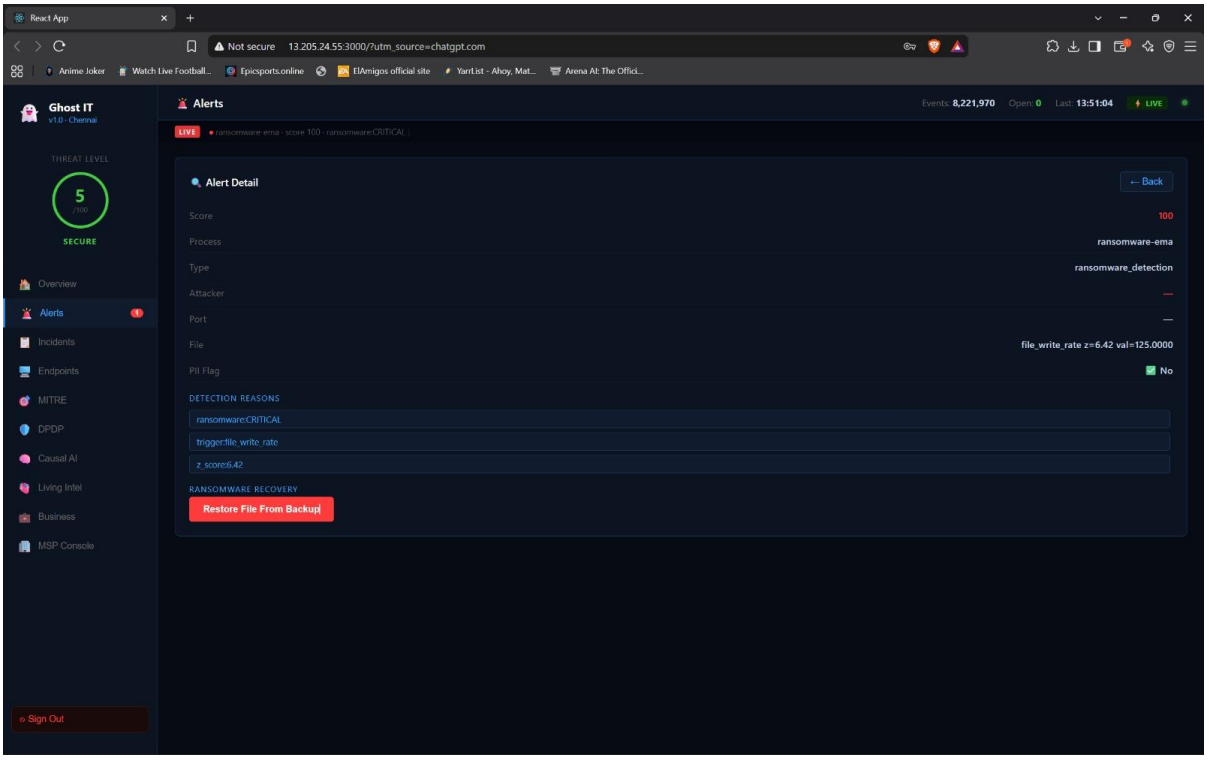



**Figure 6.2 Alert-detail output showing a ransomware-labelled alert and recovery control; supplied by the project team (23 October 2026).**

### 6.5 Acceptance criteria and measurement gaps

A future validation run should record the commit, host model, Linux distribution and kernel, CPU and memory, agent configuration, dataset or attack scenario, service versions, run command, start and stop times, raw output, and pass/fail decision. Detection quality should be measured on a documented benign and malicious corpus, with duplicate-event policy and class definitions fixed before evaluation.

| Measure | Recommended evidence | Current report status |
|---|---|---|
| Detection quality | Per-technique precision, recall, false-positive rate and confidence intervals. | No reproducible metric found. |
| Latency and capacity | Event-to-alert p50/p95/p99, sustained event rate and queue lag. | Targets/scripts exist; measured result not established. |
| Endpoint overhead | CPU, RSS, event drops and application impact across idle and load. | Acceptance checks are defined; no verified measurements. |
| Reliability | Restart, network-loss, storage-full and recovery tests with loss accounting. | Code contains retry/health mechanisms; recovery outcome unverified. |
| Operational safety | Human approval, rollback and audit evidence for each response tier. | Decision code exists; endpoint action path not validated here. |

# CHAPTER 7
# SECURITY, PRIVACY AND LIMITATIONS

## 7.1 Security and privacy analysis

Security telemetry can reveal user names, process activity, file locations, command-line arguments, network destinations and security incidents. Such records should be treated as sensitive. A deployment should provide a documented collection notice, least-privilege roles, authenticated transport, encryption at rest where required, purpose-limited retention, access logging and a tested deletion path. This report does not determine legal applicability or certify compliance with Indian law.

The Linux collector requires elevated privileges to attach kernel instrumentation, increasing the impact of a compromised agent or update path. The inspected source includes a TCP forwarding path without visible transport encryption. The dashboard API has session authentication and customer filters, but repository comments state that tenant identity is inferred from configured address ranges and that per-agent customer tagging is a longer-term approach. Strong isolation between customer data is therefore not established.

The repository also contains deployment-specific settings and fallback secrets in source or Compose configuration, including a Grafana administrator credential and an internal API-key fallback. Values are not reproduced here. They should be rotated and moved to a managed secret source. Public endpoint URLs and host-mounted paths should be parameterized, and privileged container capabilities should be minimized.

The response governor represents a useful safety boundary, but high-impact action integrations need authorization, dry-run mode, explicit customer policy, a kill switch, audit records and recovery testing. Deception components need network-isolation review before deployment. Model and agent artefact signing can support supply-chain verification, but signatures alone do not prove safe behaviour or a secure build.

### 7.2 Limitations and open risks

- The current assessment is based on source and configuration at one repository commit; it is not a production installation or independent penetration test.
- No reproducible detector-quality, throughput, latency, resource or customer-usability measurements are included in this report.
- The project documentation and code carry inconsistent status signals for the causal engine, honeypots, Windows agent and Python runtime versions.
- The root start script launches services but requires a separate privileged Linux-agent build and start step.
- Deployment configuration includes host-specific paths and credential handling that require hardening.
- The test inventory contains live-system scripts and one frontend starter test; automated CI evidence for the full suite is absent from the reviewed workflow files.
- The technical-debt register lists incomplete telemetry and detector gaps, including JA4, email phishing, T1105 context and T1110 cross-event tracking.
- The register also marks HA, per-agent mutual TLS, a secrets manager and mature control-plane features as deferred; multi-customer isolation therefore needs a dedicated security design.

### 7.3 Ethical and operational considerations

Monitoring should be proportionate to a clear security purpose and communicated to affected users. Collection of personal or business data should be minimized and retained only for a defined duration. Anomalies should trigger investigation rather than automatic assumptions of misconduct. Operators need a way to challenge incorrect alerts and correct baseline data.

Response automation must account for business criticality. Disabling a process, isolating a host or rolling back files can interrupt essential work. The system should start in observe-only mode, require explicit enablement for each action tier, and preserve a human override. Evaluation should include false positives on ordinary SME workflows, not only simulated attacks.

**Table 7.1 Security and operational risks**

| Risk | Potential impact | Mitigation for the next release |
|---|---|---|
| Privileged endpoint agent | A collector compromise or defect can affect the host. | Minimize capabilities, sign releases, validate updates and test supported kernels. |

| Risk | Potential impact | Mitigation for the next release |
|---|---|---|
| Unprotected or misdirected event transport | Telemetry exposure or spoofed events. | Use mutually authenticated encryption, bind trusted identities and rotate credentials. |
| Weak tenant boundaries | One customer may access another customer's events. | Use per-agent identity and storage partitioning; add adversarial isolation tests. |
| Sensitive telemetry retention | Exposure of user and business activity. | Document fields and retention, restrict access and verify erasure. |
| False positive response | Business interruption or data loss. | Default to observe-only, require review for high impact and prove rollback. |
| Incomplete detection inputs | Blind spots for documented attack behaviours. | Close telemetry gaps and publish a versioned, evidence-based coverage map. |

# CHAPTER 8
# CONCLUSION AND FUTURE WORK

## 8.1 Conclusion

This final-year project report presents GhostLayer as a repository-backed prototype for an adaptive cyber-monitoring system aimed at Indian SMEs. The current source has a substantial technical foundation: a Linux eBPF collection path, Python event bridge, TCP ingestion, DuckDB/Parquet storage, Redis Streams, detection modules, behavioural scoring, threshold calibration, dashboard services, canary components and a response-governance layer.

The repository also makes its limits visible. Several implementation areas have conflicting status documentation, the Linux agent is launched separately with elevated privileges, important detection inputs are missing, tenant isolation and secret management need hardening, and the checked-in test artefacts do not provide reproducible performance or detection-quality results for this report. Accordingly, the project can be described as an integrated prototype codebase, but not as a validated production cyber-immune service.

The main project outcome is a traceable design and implementation assessment that can guide the next engineering and evaluation cycle. Claims of improved detection or low operational burden should be made only after the proposed tests have been run on documented data and realistic SME workloads.

## 8.2 Prioritised future work

1. Reconcile component READMEs, deployment versions and status markers; publish one supported architecture and build matrix.
2. Create a reproducible Linux integration environment and make safe, deterministic tests run in CI.
3. Harden the deployment: remove fallback secrets, parameterize endpoints, secure event transport, minimize container privileges and review public exposure.
4. Define per-agent identity and enforce tenant separation in storage, APIs, logs and backup/restore flows.
5. Measure detection quality and endpoint overhead on labelled benign and adversarial datasets; publish thresholds, limitations and run metadata.

6. Implement missing telemetry inputs and stateful detection identified in the technical-debt register, then update a versioned ATT&CK coverage map.
7. Validate adaptive baselines against contamination and workload drift; make learning reversible and operator-visible.
8. Test response actions in dry-run and isolated environments, with human approval, auditability, rollback and recovery evidence.
9. Complete honeypot isolation, Windows agent validation, observability panels and retention controls only after requirements and test criteria are agreed.

### 8.3 Closing statement

GhostLayer provides an ambitious academic foundation for local and adaptive security monitoring. Its next milestone should be evidence: a reproducible deployment, measured effectiveness, secure multi-customer boundaries and transparent operating procedures that an SME or service provider can inspect and trust.

## APPENDIX A: REPOSITORY EVIDENCE AND COMPONENT REVIEW

Table A.1 maps the main report claims to source files in the reviewed commit. The table is a navigation aid; it does not replace a source review or runtime test.

**Table A.1 Claim-to-source traceability**

| Report claim | Repository evidence | Status used in this report |
|---|---|---|
| Linux event collection | agent/ebpf/src/ghost_agent.bpf.c; agent/ebpf/src/ghost_agent.c; agent/ebpf/python/agent.py; agent/ebpf/Makefile | Source implemented; runtime compatibility unverified. |
| Container deployment | docker-compose.yml; docker/Dockerfile.*; start.sh | Service definitions present; production deployment not asserted. |
| Ingestion and storage | pipeline/server_v2.py; pipeline/storage/db.py | Code path present; throughput and recovery unmeasured. |
| Rule and behavioural detection | detection/engine.py; detection/behavioral/*; detectors/* | Modules present; detection quality unverified. |
| Adaptive and response logic | causal-engine/adaptive_thresholds.py; autonomous_response.py; safety_governor.py | Decision code present; full integration unverified. |
| Dashboard and tenancy | dashboard/api/server.py; dashboard/api/tenancy.py; dashboard/frontend/src/App.js | Routes and UI present; security assurance incomplete. |
| Known gaps | docs/TECHNICAL_DEBT.md | Explicit deferred work and unresolved capabilities. |
| Test intentions | tests/*.py; dashboard/frontend/src/App.test.js | Scripts present; execution outputs not established. |

### A.1 Test inventory

The repository test scripts should be run only in an isolated, disposable Linux environment that matches the documented service setup. Before execution, review each script for host writes, long-running loops, network calls and assumptions about the running agent. Capture logs and test-data provenance. A future report should attach the raw outputs and summarize only results reproduced from that run.

The 14-check acceptance script contains checks that query local services, inspect kernel logs, load model artefacts, sample resource use and import gRPC protocol code. Several checks use machine-specific paths and conditions. A controlled test harness should replace those assumptions with configuration and fixtures. The default React starter assertion should be replaced with tests for sign-in, alert rendering, filtering, tenant scope and failure handling.

### A.2 Repository snapshot

Repository: https://github.com/K-v234/ghostlayer-ghostit.git

Branch: main | Reviewed commit: 4dfb877c5aaad23e456fa4bbd0c0bdba761b2fdb | Commit date: 25 August 2026 | Accessed: 22 October 2026.

The repository is the technical source of truth for the claims in this report. Files and status documentation may change after the reviewed commit.

### A.3 Component-by-component source review

This appendix expands the implementation-status summary into a source-oriented review. “Present in source” means that relevant code or configuration exists in the reviewed commit. It does not mean that the feature was compiled, deployed, exercised on a representative host, or accepted as production-ready. The path for each component is provided so that an evaluator can repeat the inspection against the cited commit.

#### A.3.1 Repository and deployment entry points

Repository paths: README.md, docker-compose.yml, start.sh, docker/Dockerfile.*

The repository is organized as a multi-service prototype with container definitions for the pipeline, detection service, dashboard API and frontend, causal service, canary service, Redis and observability. The start script launches the container stack, while the Linux kernel agent requires a separate build and elevated start procedure. This separation is operationally important: starting Compose alone is not evidence that endpoint telemetry is being collected.

Assessment boundary: The root README and Dockerfiles state different Python versions. The container bases use Python 3.11 while the README describes 3.14. A supported build matrix and one maintained entry-point procedure are needed before the deployment can be reproduced consistently.

#### A.3.2 Linux eBPF collector

Repository paths: agent/ebpf/src/ghost_agent.bpf.c, ghost_agent.c, ghost_agent.h, agent/ebpf/Makefile

The BPF source defines handlers for selected process, file, authentication, network and related events, with BPF maps for configuration and small caches. A userspace loader consumes ring-buffer records. The Makefile refers to kernel BTF data and CO-RE-style compilation. These are source-level indicators of an implemented Linux collection path.

Assessment boundary: The BPF source itself notes that only a subset of target hooks is implemented. The C event identifiers and the actual attached hook inventory are not interchangeable. A kernel and distribution compatibility matrix, event-loss counters and independently captured build/load logs are absent from the evidence reviewed for this report.

#### A.3.3 Event record contract

Repository paths: agent/ebpf/src/ghost_agent.h, agent/ebpf/python/events.py, pipeline/storage/db.py

The C header describes a richer event structure and identifier range than the Python event enum, which currently declares a small basic set. Python records include timestamps, process and user identifiers, event type, command, file and network fields. The pipeline storage schema contains additional scoring and alert columns.

Assessment boundary: This shows a schema-version boundary that should be made explicit. Without a versioned contract and compatibility tests, a new C event kind may be silently misinterpreted or dropped by the Python bridge, detection service or persistence layer. A future contract should define field types, optionality, units, encoding, and unknown-field behaviour.

#### A.3.4 Python event bridge and local scorer

Repository paths: agent/ebpf/python/agent.py, scorer.py, events.py

The bridge reads newline-delimited records from its input, applies a fixed local score function, filters low scores, marks alerts at a higher threshold, batches records and sends them over a socket. The source defines a batch size and periodic flush, and includes fallback output and watchdog behaviour.

Assessment boundary: The scorer is a deterministic heuristic, not a trained model or measured risk estimator. Fixed names, paths, ports and command classes may not transfer to every SME workload. The bridge path also requires explicit transport protection and authentication before it is placed on an untrusted network. Its field mapping needs tests against the C producer and pipeline consumer.

#### A.3.5 Ingestion listener and deployed pipeline

Repository paths: pipeline/ingestion/server.py, pipeline/server_v2.py, docker-compose.yml

The repository contains a simple TCP newline-JSON ingestion implementation and a larger server_v2 pipeline. Compose selects server_v2.py as the pipeline service entry point. The latter defines event routes, buffering, stream operations, storage paths and operational endpoints.

Assessment boundary: The alternate listener should be documented as such to avoid an inaccurate description of the active Compose route. The large service has many API surfaces; endpoint presence does not establish authorization coverage, stable contracts or load capacity. Integration tests need to confirm that the agent's exact JSON payload reaches the deployed service.

#### A.3.6 Queueing and durable storage

Repository paths: pipeline/server_v2.py, pipeline/storage/db.py

The source describes an in-memory hot buffer, Redis Streams, Parquet output, DuckDB queries and a write-ahead log recovery path. The event database schema includes event time, received time, process identity, event fields, score, alert flag and reason text, with indexes for common queries.

Assessment boundary: The report does not have captured evidence for loss-free recovery, duplicate handling, ordering guarantees, file rotation or scheduled retention. The database helper contains cleanup logic, but the existence of a cleanup function does not prove that an approved retention schedule is configured and running.

#### A.3.7 Deterministic rules and window aggregation

Repository paths: detection/rules.py, detection/aggregator.py

Rule code evaluates explicit indicators such as sensitive file access, suspicious shells and selected command patterns. The aggregator maintains short windows and compares counts against behaviour-specific limits. This design offers readable reasons and is suitable for deterministic unit tests.

Assessment boundary: Rules are only as complete as the event fields supplied to them. Narrow string matching can create false positives or miss equivalent behaviour. Window state requires defined expiry and restart semantics. Per-rule false-positive review and tested event fixtures are not presented as results in this report.

#### A.3.8 Behavioural feature extraction

Repository paths: detection/behavioral/features.py

The feature extractor defines a locked 17-feature vector over short event windows. It uses process, file, connection and timing observations to summarize behaviour for later baseline and anomaly-scoring components.

Assessment boundary: Several features are explicit zero placeholders because their source signals are not integrated. Outbound network bytes are estimated from a connection count rather than measured from byte counters. The reported feature vector is therefore a mixed set of direct observations, derived proxies and unavailable inputs; model performance must not be inferred from the feature count alone.

#### A.3.9 Baseline and anomaly scoring

Repository paths: detection/behavioral/baseline.py, detection/behavioral/isolation_forest.py, detection/behavioral/engine.py

Baseline code updates per-entity and population statistics with EWMA-style logic. The behavioural engine combines feature windows with a loaded or trained Isolation Forest and other invariant or cross-entity checks. The repository also contains model pipeline and metadata files.

Assessment boundary: Training provenance, representative SME workloads, labelled evaluation sets, calibration, drift tests and model-quality metrics are not established by the artefacts inspected here. Persisted baselines can be contaminated by malicious activity; reset, freeze, rollback and operator review controls must be validated.

#### A.3.10 Adaptive threshold calibration

Repository paths: causal-engine/adaptive_thresholds.py

The threshold component records score observations, bounds the retained window, requires a minimum sample count before calibration and computes a percentile-derived result within configured lower and upper limits. It has a conservative fallback threshold when data is insufficient.

Assessment boundary: This is a defined calibration algorithm, but the repository does not prove that the service producing a score and the component consuming it share an end-to-end validated path. Contamination, seasonality, sparse data, customer-specific baselines and reset procedures require controlled tests.

#### A.3.11 Response decision and safety governor

Repository paths: causal-engine/autonomous_response.py, causal-engine/safety_governor.py

Response modules represent tiered proposals, and the safety governor can downgrade a proposal or request confirmation under selected conditions

involving criticality, blast radius and false-positive history. This provides a useful decision boundary in source.

Assessment boundary: A policy function is not the same as a working containment integration. The endpoint action, authorization check, audit event, human confirmation, cancellation and rollback path need independent verification. Until those tests are complete, high-impact response should remain disabled or manual.

#### A.3.12 Dashboard API and customer scoping

Repository paths: dashboard/api/server.py, dashboard/api/tenancy.py

The API exposes authentication and operational routes. The tenancy helper maps a user to a customer and filters by configured address ranges, which is a source-level attempt at scoping.

Assessment boundary: The source relies on address-based mapping and a default internal identity rather than a demonstrated cryptographic per-agent identity. Hard-coded credential material exists in repository configuration; this report deliberately does not reproduce it. Secrets must be rotated and moved out of source, while adversarial cross-tenant tests must verify every read and write route.

#### A.3.13 Web interface

Repository paths: dashboard/frontend/src/App.js, App.css, App.test.js

The frontend contains sign-in and dashboard components that consume API data, while the current test file still checks the default React starter content. Source presence supports a user-interface implementation claim.

Assessment boundary: The frontend test does not verify GhostLayer flows, authorization failure, empty states, slow APIs, tenant filters, keyboard access or alert evidence. UI status should therefore be stated as source present and workflow assurance incomplete.

#### A.3.14 Deception services

Repository paths: deception/canary/*, deception/honeypots/*, docker/Dockerfile.canary

Canary tokens, file monitoring and server modules are present. Honeypot sources include orchestration and isolation helpers, with comments describing alternative sandbox paths.

Assessment boundary: Component README and inline status notes conflict about completion. Network isolation, decoy-data safety, outbound restrictions, cleanup and resource limits are not proven by code presence. Deploy only in an isolated lab until those controls are independently exercised.

#### A.3.15 Windows monitoring path

Repository paths: windows-agent/src/*, windows-agent/README.md

The tree includes ETW provider, event serializer, process monitoring and forwarding source files. This is evidence of an attempted Windows path.

Assessment boundary: The README's status conflicts with source-file completion comments, and no Windows build or end-to-end output was reviewed. The report classifies this path as partial and unverified, without claiming feature parity with Linux.

#### A.3.16 Observability, CI and release artefacts

Repository paths: observability/*, .github/workflows/*, tests/*

Prometheus and Grafana configuration and release/model workflows are present. The repository includes acceptance, detector, integration, regression, performance and simulation scripts.

Assessment boundary: The workflows do not provide a reproduced full-suite result for this report, and the technical-debt file marks some panels and alerts as deferred. Acceptance criteria, artefact signing, provenance and environment metadata should be tied together in future release evidence.

## APPENDIX B: SELECTED SOURCE LISTINGS

The following short excerpts are taken directly from the reviewed repository commit. Together they occupy approximately 5–10 pages and show representative event, scoring, ingestion, rule, threshold and safety-governor paths. The excerpts make the report's descriptions auditable; they are not code added or modified for this report. Line numbers refer to the repository snapshot, and credential or signing material is excluded.

**agent/ebpf/python/events.py**

The Python event model declares the fields and event identifiers used by the bridge.

**Source listing: agent/ebpf/python/events.py, lines 1-49, repository commit 4dfb877**

```
0001                                                                                """
0002                        Ghost             IT             –             Event             Schema
0003            Type-safe      Python      mirror      of      the      C      ghost_event      struct.
0004                                                                                """
0005                        from             __future__              import             annotations
0006                    from          dataclasses          import          dataclass,          asdict
0007                             from               enum               import              IntEnum
0008                           from              typing              import              Optional
0009                                                    import                                 json
0010
0011
0012                                              class                          EventType(IntEnum):
0013                                               EXEC                                   =        1
0014                                               OPEN                                   =        2
0015                                                                    CONNECT           =           3
0016                                                    CLONE                               =         4
0017                                                          UNLINK                      =          5
0018
0019
0020                                                                                    @dataclass
0021                                              class                                 GhostEvent:
0022                        """Structured     kernel     event     from     the     eBPF     agent."""
0023                  ts:          int                                    #   Nanoseconds   since   boot
0024                                                                      pid:                        int
0025                                                                              ppid:               int
0026                                                                      uid:                        int
0027                                                                      gid:                        int
0028                       comm:     str                                          #    Process    name
0029                      type:    str                                          #    EventType    string
0030
0031                       #    Optional    fields    –    present    depending    on    event    type
0032                            file:                                      Optional[str]     =     None
0033                            args:                                      Optional[str]     =     None
0034                              flags:                                   Optional[int]     =     None
0035                              daddr:                                   Optional[str]     =     None
0036                              dport:                                   Optional[int]     =     None
0037                                family:                                Optional[int]     =     None
0038                                              clone_flags:        Optional[int]        =        None
0039
0040                                                                                   @classmethod
0041                                 def      from_dict(cls,      d:      dict)      ->      GhostEvent:
0042                                     valid    =    {k    for    k    in    cls.__dataclass_fields__}
0043                          return   cls(**{k:   v   for   k,   v   in   d.items()   if   k   in   valid})
0044
0045                                                    def        to_dict(self)        ->        dict:
0046                     return  {k:  v  for  k,  v  in  asdict(self).items()  if  v  is  not  None}
0047
```

```
0048                                                def        to_json(self)        ->        str:
0049         return json.dumps(self.to_dict())
```

**agent/ebpf/python/scorer.py**

This deterministic score function uses source-defined heuristics; its thresholds are not field-calibrated measurements.

**Source listing: agent/ebpf/python/scorer.py, lines 1-70, repository commit 4dfb877**

```
0001                                                                                                """
0002                    Ghost           IT           —           Suspicion           Scorer           v2
0003          Scores     each     event     0-100.     Fixed     false     positives     from     v1.
0004                                                                                                """
0005                         from             __future__             import             annotations
0006                           from               events               import              GhostEvent
0007
0008                          THRESHOLD_LOG                                          =              10
0009                                      THRESHOLD_ALERT                     =                  40
0010
0011              #       Exact       sensitive       file       paths       (not       directories)
0012                                      SENSITIVE_FILES                     =                    {
0013                                       "/etc/passwd",       "/etc/shadow",       "/etc/sudoers",
0014                                                          "/etc/crontab",          "/etc/hosts",
0015                                                                                                }
0016
0017        #     Sensitive     directory     prefixes     —     only     flag     writes,     not     reads
0018                                    SENSITIVE_DIRS_WRITE_ONLY                 =                 {
0019                                               "/root/.ssh",        "/etc/ssh",        "/var/log",
0020                                                                                                }
0021
0022                #         Always         suspicious         regardless         of         read/write
0023                                      ALWAYS_SENSITIVE                      =                   {
0024                                             "/tmp",        "/dev/shm",        "/proc/self/mem",
0025                                                                                                }
0026
0027                                      SUSPICIOUS_PORTS                    =                    {
0028                                                 4444,         1337,         31337,         8888,
0029                                                  6666,         6667,         6668,         6669,
0030                                                                              9001,             9030,
0031                                                                                                }
0032
0033        NO_NET_PROCS     =     {"bash",     "sh",     "python3",     "python",     "perl",     "ruby"}
0034                  NO_FORK_PROCS          =          {"nginx",          "apache2",          "sshd"}
0035
0036                #         Exact         binary         names         —         not         substrings
0037                                        ATTACKER_TOOLS                      =                     {
0038                                     "nmap",       "nc",       "netcat",       "curl",       "wget",
0039                                        "socat",       "ncat",       "tcpdump",       "wireshark",
0040                                                "msfconsole",        "msfvenom",        "sqlmap",
0041                                                                                                }
0042
0043                                  #                Exact                shell                names
0044             SHELLS      =      {"bash",      "sh",      "dash",      "zsh",      "fish",      "ksh"}
0045
0046                         #            Interpreters            that            exec            things
0047        INTERPRETERS    =    {"python3",    "python",    "python2",    "perl",    "ruby",    "php"}
0048
0049
0050                         def            _basename(path:            str)            ->            str:
0051                        return    path.rstrip("/").rsplit("/",    1)[-1]    if    path    else    ""
0052
0053
0054              def       score(event:       GhostEvent)       ->       tuple[int,       list[str]]:
0055                                                                    s            =            0
0056                                                                reasons           =           []
0057
0058                                                  if         event.type         ==         "exec":
0059                                                                          s        +=        5
0060                                              fname     =     _basename(event.file     or     "")
0061
0062                                                           if      fname      in      SHELLS:
0063                                                                                  s      +=      20
```

```
0064                                                    reasons.append("shell_spawned")
0065
0066                                                if      fname      in      INTERPRETERS:
0067                                                                    s      +=      15
0068                                                      reasons.append("interpreter_exec")
0069
0070          # Exact match only — no substring
```

**agent/ebpf/python/agent.py**

This excerpt shows input parsing and the beginning of the bridge path; source presence does not establish a live endpoint integration.

**Source listing: agent/ebpf/python/agent.py, lines 1-80, repository commit 4dfb877**

```
0001                                            #!/usr/bin/env                          python3
0002                                                                                      """
0003                Ghost        IT        Agent        —        Pipeline        Forwarder        v3
0004                        -            Scored            +            filtered            events
0005       -    Time-based    flush    (every    5s)    +    size-based    flush    (every    N    events)
0006                    -          Auto-reconnect          on          pipeline          disconnect
0007                                                                                      """
0008                                                    import                            sys
0009                                                   import                           json
0010                                                    import                             os
0011                                                  import                          socket
0012                                                 import                          logging
0013                                                import                          argparse
0014                                               import                         threading
0015                            from              typing              import             Optional
0016                           from             events             import            GhostEvent
0017              from       scorer       import       score,       THRESHOLD_LOG,       THRESHOLD_ALERT
0018
0019                                                                          logging.basicConfig(
0020                                                                           level=logging.INFO,
0021                       format="%(asctime)s     [ghost-agent]     %(levelname)s     %(message)s",
0022                                                                             datefmt="%Y-%m-%dT%H:%M:%S",
0023                                                                              stream=sys.stderr,
0024                                                                                          )
0025                               log                =                logging.getLogger(__name__)
0026
0027
0028                                          class                          PipelineForwarder:
0029                                 def      __init__(self,      host:      str,      port:      int,
0030                                      batch_size:  int  =  50,  flush_interval:  float  =  5.0):
0031                                     self.host                                          =    host
0032                                     self.port                                          =    port
0033                                           self.batch_size                      =     batch_size
0034                                                    self.flush_interval     =      flush_interval
0035                                               self.batch:     list                    =     []
0036                                  self.lock                                  =    threading.Lock()
0037                                               self.sock:     Optional[socket.socket]     =     None
0038                                                                                  self._connect()
0039                                                                               self._start_timer()
0040
0041                                                                      def            _connect(self):
0042                                                                                          try:
0043                                           s   =   socket.socket(socket.AF_INET,   socket.SOCK_STREAM)
0044                                                                                    s.settimeout(3)
0045                                                                      s.connect((self.host,     self.port))
0046                                                                                 self.sock      =      s
0047                                  log.info(f"Connected  to  pipeline  at  {self.host}:{self.port}")
0048                                                       except     (ConnectionRefusedError,     OSError):
0049                                  log.warning("Pipeline  unavailable  —  stdout  fallback  active")
0050                                                                                 self.sock     =     None
0051
0052                                                                           def           _start_timer(self):
0053                                     """Flush    on    timer    regardless    of    batch    size."""
0054                                                                                       def        _tick():
0055                                                                                              self.flush()
0056                                     self._timer   =   threading.Timer(self.flush_interval,   _tick)
0057                                                                              self._timer.daemon     =     True
0058                                                                                       self._timer.start()
```

```
                                self._timer    =    threading.Timer(self.flush_interval,    _tick)
                                                    self._timer.daemon      =      True
                                                                        self._timer.start()

            def   forward(self,   event:   GhostEvent,   event_score:   int,   reasons:   list):
                                                        payload      =      event.to_dict()
                                            payload["score"]               =     event_score
                                                      payload["reasons"]      =      reasons
                                    payload["alert"]          =    event_score    >=    THRESHOLD_ALERT
                                                                        with        self.lock:
                                                                  self.batch.append(payload)
                                                if    len(self.batch)    >=    self.batch_size:
                                                                    self._flush_locked()

                                                                 def            flush(self):
                                                                      with        self.lock:
                                                                        self._flush_locked()

                                                            def           _flush_locked(self):
                                                                if       not       self.batch:
                                                                                        return
        data = (json.dumps(self.batch) + "\n").encode()
```

**pipeline/storage/db.py**

The excerpt shows the storage schema and helpers used to reason about persisted event fields.

**Source listing: pipeline/storage/db.py, lines 1-70, repository commit 4dfb877**

```
                                                                                """
                  Ghost          IT          –          DuckDB          Storage          Layer
        Handles      all      database      operations      for      the      event      pipeline.
    Schema    designed    for    fast    time-series    queries    +    detection    engine    access.
                                                                                """
                        from             __future__             import             annotations
                                                    import                              duckdb
                                                   import                             logging
                                                      import                                os
                           from               typing               import               Optional

                               log                =                 logging.getLogger(__name__)

        DB_PATH      =      os.environ.get("GHOSTIT_DB_PATH",      "/var/lib/ghostit/events.db")


                                                 class                            EventStore:
                                                                                        """
                                             Thread-safe        DuckDB        event        store.
                      Single     writer,     multiple     readers     via     DuckDB's     MVCC.
                                                                                        """

                           def      __init__(self,      db_path:      str      =      DB_PATH):
                                      os.makedirs(os.path.dirname(db_path),     exist_ok=True)
                                                        self.db_path       =       db_path
                                                 self.conn      =      duckdb.connect(db_path)
                                                                    self._init_schema()
                                           log.info(f"EventStore    initialized    at    {db_path}")

                                                                 def           _init_schema(self):
                                                                               self.conn.execute("""
                                                CREATE    TABLE    IF    NOT    EXISTS    events    (
                                               id                                  UBIGINT   PRIMARY   KEY,
                      ts               UBIGINT  NOT  NULL,            --  nanoseconds  since  boot
                           received_at  TIMESTAMP  DEFAULT  now(),  --  wall  clock  insert  time
                                                pid                              UINTEGER   NOT   NULL,
                                                  ppid                           UINTEGER   NOT   NULL,
                                                uid                              UINTEGER   NOT   NULL,
                                                gid                              UINTEGER   NOT   NULL,
                                                comm                            VARCHAR     NOT   NULL,
                                                type                            VARCHAR     NOT   NULL,
                                                  score                          USMALLINT   NOT   NULL,
                                                  alert                         BOOLEAN     NOT   NULL,
```

```
                        reasons       VARCHAR[],                    -- array of reason strings
                                                        file                            VARCHAR,
                                                        args                            VARCHAR,
                                                          flags                         INTEGER,
                                                          daddr                         VARCHAR,
                                                        dport                         USMALLINT,
                                                          family                      USMALLINT,
                                                                              clone_flags     UBIGINT
                                                                                              )
                                                                                             """)

                                                  #     Indexes     for     common     query     patterns
                                                                                      self.conn.execute("""
                                                   CREATE    INDEX    IF    NOT    EXISTS    idx_events_ts
                                                                                ON     events     (ts     DESC)
                                                                                             """)
                                                                                      self.conn.execute("""
                                                CREATE    INDEX    IF    NOT    EXISTS    idx_events_alert
                                                                     ON     events     (alert,     ts     DESC)
                                                                                             """)
                                                                                      self.conn.execute("""
                                                   CREATE    INDEX    IF    NOT    EXISTS    idx_events_pid
                                                                           ON     events     (pid,     ts     DESC)
                                                                                             """)
                                                                                      self.conn.execute("""
                                                  CREATE    INDEX    IF    NOT    EXISTS    idx_events_comm
            ON events (comm, ts DESC)
```

**detection/rules.py**

These rules illustrate direct event matching and their dependency on the upstream event vocabulary.

**Source listing: detection/rules.py, lines 1-80, repository commit 4dfb877**

```
                                                                                                """
                      Ghost            IT            —            Rule-Based            Detector
       Fast     pattern     matching     on     individual     events     and     small     windows.
       Rules     are     ordered     by     severity.     First     match     wins     per     event.
                                                                                                """
                         from             __future__             import             annotations
                         from             dataclasses              import              dataclass
                            from              typing               import               Optional


                                                                                              @dataclass
                                                    class                                     Detection:
                                                rule_id:                                            str
              severity:           str                        #   critical   |   high   |   medium
                                      title:                                                    str
                                                                    description:                str
                          confidence:         int                                     #     0-100
                                              evidence:                                  list[dict]
       detection_method: str = "deterministic_rule"   # deterministic_rule | statistical_ema |
ml_isolation_forest             |             ml_graphsage             |             deception_trigger
                                                 schema_version:         int         =         1


              #       ------------------------------------------------------------------        #
  # Individual event rules                                                                      #
              #       ------------------------------------------------------------------        #

                def        check_event(event:        dict)        ->        Optional[Detection]:
                                                                                                """
                                Check     a     single     event     against     all     rules.
                          Returns     Detection     if     a     rule     fires,     None     otherwise.
                                                                                                """
                                           comm              =       event.get("comm",        "")
                                               type_         =       event.get("type",        "")
                                          file_       =       event.get("file")       or       ""
                                          daddr       =       event.get("daddr")      or       ""
                                           dport      =       event.get("dport")      or       0
                                                score        =       event.get("score",       0)
```

```
0038
0039                                # R001 — Canary hit (always critical)
0040                                if type_ == "canary_hit":
0041                                    return Detection(
0042                                        rule_id = "R001",
0043                                        severity = "critical",
0044                                        title = "Canary Token Triggered",
0045                                        description = f"Attacker accessed decoy asset: {file_}",
0046                                        confidence = 100,
0047                                        evidence = [event],
0048                                    )
0049
0050                                # R002 — Shell reading shadow file
0051                if comm in ("bash","sh","dash","zsh") and "/etc/shadow" in file_:
0052                                    return Detection(
0053                                        rule_id = "R002",
0054                                        severity = "critical",
0055                                        title = "Shadow File Access",
0056                                        description = f"{comm} read /etc/shadow — credential theft attempt",
0057                                        confidence = 95,
0058                                        evidence = [event],
0059                                    )
0060
0061                                # R003 — Reverse shell ports
0062                if type_ == "connect" and dport in (4444,1337,31337,9001,6666,8888):
0063                                    return Detection(
0064                                        rule_id = "R003",
0065                                        severity = "critical",
0066                                        title = "Reverse Shell Connection",
0067                description = f"{comm} connected to {daddr}:{dport} — known reverse shell port",
0068                                        confidence = 90,
0069                                        evidence = [event],
0070                                    )
0071
0072                        # R004 — Script interpreter making outbound connection
0073        if type_ == "connect" and comm in ("perl","ruby","php","python3","python","lua","node")
and daddr not in ("127.0.0.1", "::1", "", "1.0.0.127", "0.0.0.127", "11.0.0.127", "10.0.0.127"):
0074                                    return Detection(
0075                                        rule_id = "R004",
0076                                        severity = "high",
0077                                    title = "Interpreter Outbound Connection",
0078                        description = f"{comm} made outbound TCP connection to {daddr}:{dport}",
0079                                        confidence = 75,
0080                evidence = [event],
```

**causal-engine/adaptive_thresholds.py**

The threshold component demonstrates bounded percentile calibration and a minimum-sample fallback.

**Source listing: causal-engine/adaptive_thresholds.py, lines 1-75, repository commit 4dfb877**

```
0001                                        #!/usr/bin/env python3
0002                                                """
0003                    Ghost IT — Adaptive Threshold Calibration
0004
0005          Architectural gap this closes: detection thresholds (C15's entropy
0006         delta, C2's z-scores, C14's confidence cutoffs) are fixed constants
0007          chosen once, globally, for every deployment. A quiet accounting
0008        firm and a noisy dev shop running constant compilers/build tools have
0009         wildly different baseline activity levels -- a fixed threshold is
0010        either too sensitive (drowns the quiet deployment in false positives)
0011       or too blind (misses real signal on the noisy one, since its "normal"
0012        already looks like what the fixed threshold was tuned to catch).
0013
0014          This module continuously observes each deployment's own event-score
0015            distribution and computes a locally-appropriate threshold using
0016         percentile-based calibration -- the threshold that would have flagged
0017        only the top N% most unusual events THIS SPECIFIC DEPLOYMENT actually
0018            produced, rather than a number chosen in the abstract.
0019                                                """
0020                            from __future__ import annotations
0021                                            import os
```

```
0022                                                    import                                  time
0023                                                  import                                  logging
0024                                                 import                                 threading
0025                                                import                                statistics
0026                                                   import                                  duckdb
0027
0028                                log                 =                 logging.getLogger(__name__)
0029
0030                    THRESHOLDS_DB_PATH           =           os.environ.get("THRESHOLDS_DB_PATH",
0031                                os.path.expanduser("~/ghostlayer/data/adaptive_thresholds.duckdb"))
0032
0033        #    How    many    recent    score    observations    to    keep    per    pillar    for
0034       #   calibration   --   large   enough   for   statistical   stability,   small   enough
0035      #   that   the   baseline   adapts   to   genuinely   new   normal   behavior   (e.g.   a
0036        #    new    backup    tool    being    installed)    within    a    reasonable    window.
0037                                      WINDOW_SIZE                    =                       2000
0038
0039        #    Target    percentile:    the    threshold    is    set    so    that    roughly    this
0040       #   fraction   of   a   deployment's   OWN   observed   activity   would   be   flagged
0041       #   --   i.e.   "the   top   2%   most   unusual   things   THIS   deployment   does."
0042        #    Tunable    per    pillar    since    some    (deception)    should    have    near-zero
0043        #    tolerance    while    others    (behavioral)    genuinely    need    more    headroom.
0044                                   DEFAULT_TARGET_PERCENTILE                =                  98.0
0045
0046      #   Hard   floor/ceiling   so   a   deployment's   own   noise   can't   calibrate   the
0047        #    threshold    into    something    nonsensical    (e.g.    a    deployment    that's
0048        #    ALREADY    compromised    and    constantly    noisy    shouldn't    calibrate    its
0049                 #          own          threshold          up          to          "normal").
0050                                      MIN_THRESHOLD                    =                    30
0051                                      MAX_THRESHOLD                    =                    95
0052
0053                                              class                          AdaptiveThresholds:
0054                          def     __init__(self,     db_path:     str     =     THRESHOLDS_DB_PATH):
0055                                           os.makedirs(os.path.dirname(db_path),     exist_ok=True)
0056                                                    self._lock      =      threading.Lock()
0057                                                 self.conn      =      duckdb.connect(db_path)
0058                                                                          self._init_schema()
0059                                     log.info(f"AdaptiveThresholds    initialized:    {db_path}")
0060
0061                                                        def           _init_schema(self):
0062                                                                    self.conn.execute("""
0063                                          CREATE    TABLE    IF    NOT    EXISTS    observations    (
0064                                                                        pillar    VARCHAR    NOT    NULL,
0065                                                                     score       DOUBLE    NOT    NULL,
0066                                                               ts                 DOUBLE    NOT    NULL
0067                                                                                                    )
0068                                                                                                   """)
0069                                                                                  self.conn.execute(
0070                              "CREATE  INDEX  IF  NOT  EXISTS  idx_pillar  ON  observations(pillar)")
0071
0072                                   def     observe(self,     pillar:     str,     score:     float):
0073                                                                                                   """
0074                           Record  a  raw  score  observation  for  this  pillar,  regardless  of
0075          whether it crossed any threshold -- calibration needs to see
```

**causal-engine/safety_governor.py**

The governor is decision logic; it does not prove endpoint action, approval workflow or rollback integration.

**Source listing: causal-engine/safety_governor.py, lines 1-79, repository commit 4dfb877**

```
0001                                                #!/usr/bin/env                          python3
0002                                                                                                   """
0003                           Ghost            IT            —            Safety            Governor
0004
0005        The    gap    this    closes:    Autonomous    Response's    five    safety    gates    check
0006        confidence    and    cross-pillar    agreement,    but    nothing    checks    the    actual
0007         real-world    consequence    of    the    action.    This    governor    sits    between
0008         Autonomous    Response's    decision    and    any    real    execution,    consulting
0009         the    World-Model    to    check    blast    radius    and    business    criticality
0010                before        compiling        a        final,        deterministic        verdict.
0011                                                                                                   """
```

```
0012                            from              __future__              import              annotations
0013                                                                 import                                 logging
0014
0015                                  log                 =                 logging.getLogger(__name__)
0016
0017                                CRITICALITY_DOWNGRADE_THRESHOLD                 =                 80
0018                                BLAST_RADIUS_DOWNGRADE_THRESHOLD                  =                 5
0019
0020      def   govern(proposed_tier:   int,   proposed_action:   str,   world_model_assessment:   dict,
0021                                        false_positive_history_count:   int   =   0)   ->   dict:
0022                     criticality_score    =    world_model_assessment.get("criticality_score",    0)
0023                        blast_radius    =    world_model_assessment.get("blast_radius_count",    0)
0024                                                                reasons           =           []
0025                                                           final_tier         =         proposed_tier
0026                                                         final_action         =         proposed_action
0027
0028                                if     criticality_score     >=     CRITICALITY_DOWNGRADE_THRESHOLD:
0029                                                                                        reasons.append(
0030                         f"Target  has  high  business  criticality  ({criticality_score}/100)  --  "
0031                            f"downgrading  from  tier  {proposed_tier}  to  reduce  business  risk"
0032                                                                                        )
0033                                                          final_tier     =     min(final_tier,     2)
0034                  final_action  =  "suspend_process"  if  final_tier  >=  2  else  "throttle_network"
0035
0036                                     if     blast_radius     >=     BLAST_RADIUS_DOWNGRADE_THRESHOLD:
0037                                                                                        reasons.append(
0038                                    f"Blast  radius  of  {blast_radius}  dependent  processes  --  "
0039                                   f"isolating  would  affect  {blast_radius}  additional  processes"
0040                                                                                        )
0041                                                          final_tier     =     min(final_tier,     2)
0042
0043                                               if       false_positive_history_count       >=       3:
0044                                                                                        reasons.append(
0045                           f"This  entity  type  has  {false_positive_history_count}  prior  false  "
0046                               f"positives  --  requiring  human  confirmation  before  proceeding"
0047                                                                                        )
0048                                                                                    return        {
0049                                                            "verdict":    "requires_human_confirmation",
0050                              "proposed_tier":  proposed_tier,  "proposed_action":  proposed_action,
0051                                                                                "reasons":     reasons,
0052                                                                                        }
0053
0054                                                                          if          not          reasons:
0055                       reasons.append("No  business-risk  factors  found  --  proceeding  as  proposed")
0056
0057                                                                                      return            {
0058                         "verdict":  "approved"  if  final_tier  ==  proposed_tier  else  "downgraded",
0059                                            "final_tier":    final_tier,    "final_action":    final_action,
0060                                 "original_tier":   proposed_tier,   "original_action":   proposed_action,
0061                                                                              "reasons":       reasons,
0062                                                                                        }
0063
0064                                     if               __name__               ==               "__main__":
0065                                                                        logging.basicConfig(level=logging.INFO)
0066
0067          print("=== Scenario 1: high-confidence ransomware detection on a THROWAWAY script ===")
0068             r1  =  govern(3,  "isolate_host",  {"criticality_score":  0,  "blast_radius_count":  0})
0069                                                                  print(f"                    {r1}\n")
0070
0071         print("=== Scenario 2: identical detection, but on a CRITICAL database with 5 dependents
===")
0072            r2  =  govern(3,  "isolate_host",  {"criticality_score":  100,  "blast_radius_count":  5})
0073                                                                  print(f"                    {r2}\n")
0074
0075          print("=== Scenario 3: same critical target, but this process TYPE has 3 prior false
positives                                                                                   ===")
0076            r3  =  govern(3,  "isolate_host",  {"criticality_score":  100,  "blast_radius_count":  5},
false_positive_history_count=3)
0077                                                                  print(f"                    {r3}\n")
0078
0079          print("=== Result: identical raw detection confidence, THREE genuinely different final
verdicts based on real business context ===")
```

## APPENDIX C: REQUIREMENTS AND VERIFICATION PLAN

This appendix gathers the most useful acceptance requirements, verification protocols and a staged pilot plan. These are proposed engineering checks informed by repository structure; they are not statements that a test has passed. Tests listed here were not executed for this report.

### C.1 Requirements catalogue

#### FR-01 Kernel event capture

Source or stakeholder need: The Linux endpoint path.

Proposed acceptance evidence: Collect the declared event classes on each supported kernel; attach, detach, and unsupported-hook outcomes are logged and counted.

#### FR-02 Event type mapping

Source or stakeholder need: C and Python event definitions.

Proposed acceptance evidence: Every C event type has an explicit Python and backend mapping or a counted, safely retained unknown-event path.

#### FR-03 Process context

Source or stakeholder need: Process identifiers and command fields.

Proposed acceptance evidence: A process record preserves PID, parent PID and event time with a documented handling rule for PID reuse.

#### FR-04 File event context

Source or stakeholder need: File and flags fields.

Proposed acceptance evidence: File activity contains only declared path metadata; omitted or truncated paths are distinguishable from empty values.

#### FR-05 Network event context

Source or stakeholder need: Destination and port fields.

Proposed acceptance evidence: Network records define address family, address format, port byte order and behaviour when fields are unavailable.

#### FR-06 Batch forwarding

Source or stakeholder need: Python event bridge.

Proposed acceptance evidence: A batch is transmitted once or retained for a bounded retry; duplicate and loss behaviour is measurable.

**FR-07 Transport identity**

Source or stakeholder need: Agent-to-pipeline path.

Proposed acceptance evidence: Each endpoint is authenticated and traffic is encrypted before a production network path is approved.

**FR-08 Input validation**

Source or stakeholder need: TCP ingestion service.

Proposed acceptance evidence: Malformed, oversized, partial and unknown-version records are rejected or quarantined without stopping the listener.

**FR-09 Queue observability**

Source or stakeholder need: Redis Streams and pipeline buffer.

Proposed acceptance evidence: Queue depth, oldest-record age, consumer lag, retries and dead-letter counts are exposed.

**FR-10 Durable event write**

Source or stakeholder need: DuckDB, Parquet and WAL code.

Proposed acceptance evidence: An acknowledged event can be recovered after a controlled restart, with duplicate and loss accounting.

**FR-11 Retention enforcement**

Source or stakeholder need: Database cleanup helpers and deployment schedule.

Proposed acceptance evidence: A configured retention period is applied automatically and records of execution are retained for audit.

**FR-12 Deterministic alerting**

Source or stakeholder need: Rule detector.

Proposed acceptance evidence: Each rule has an identifier, version, severity, required fields, reason, and unit tests for positive and negative cases.

**FR-13 Window correlation**

Source or stakeholder need: Behaviour aggregator.

Proposed acceptance evidence: Window length, threshold, entity key and reset behaviour are explicit and configurable.

**FR-14 Feature availability**

Source or stakeholder need: Behavioural feature extractor.

Proposed acceptance evidence: Every feature declares its source, unit, missing-data value and freshness; placeholder values are not mistaken for observations.

**FR-15 Baseline review**

Source or stakeholder need: EWMA and population baseline.

Proposed acceptance evidence: Baseline updates can be frozen, inspected, reset and restored; learning periods and source events are auditable.

**FR-16 Anomaly score explanation**

Source or stakeholder need: Isolation Forest and engine.

Proposed acceptance evidence: A model score is accompanied by version, feature availability and a plain-language explanation without presenting the score as a verdict.

**FR-17 Threshold calibration**

Source or stakeholder need: AdaptiveThresholds module.

Proposed acceptance evidence: Calibration records sample count, window, percentile, bounds and fallback state for every returned threshold.

**FR-18 Threshold rollback**

Source or stakeholder need: Adaptive state.

Proposed acceptance evidence: An operator can export, freeze and restore thresholds to a prior known-safe state.

**FR-19 Response proposal**

Source or stakeholder need: Response and governor modules.

Proposed acceptance evidence: A proposal includes reason, target, blast-radius estimate, expiry and required approval tier.

**FR-20 Human confirmation**

Source or stakeholder need: Safety governor and dashboard.

Proposed acceptance evidence: A confirmation request is bound to an authenticated operator and expires safely if unanswered.

**FR-21 Containment rollback**

Source or stakeholder need: Future endpoint action integration.

Proposed acceptance evidence: A reversible action has a documented precondition, audit record, timeout, rollback path and isolated test evidence.

**FR-22 Canary alert**

Source or stakeholder need: Canary token/file watcher.

Proposed acceptance evidence: A canary event identifies the decoy, host, time and evidence while avoiding unnecessary sensitive content.

**FR-23 Honeypot isolation**

Source or stakeholder need: Honeypot orchestrator.

Proposed acceptance evidence: A decoy is network-contained, resource-limited, disposable and unable to reach production systems by default.

**FR-24 Dashboard alert review**

Source or stakeholder need: API and React frontend.

Proposed acceptance evidence: A user can see alert reason, event time, affected endpoint, related evidence and an explicit current status.

**FR-25 Tenant-scoped access**

Source or stakeholder need: Dashboard tenancy code.

Proposed acceptance evidence: Every customer-facing query and mutation is checked against cryptographic user and agent identities.

**FR-26 Audit history**

Source or stakeholder need: Compliance and incident routes.

Proposed acceptance evidence: Sensitive reads, response approvals, configuration changes and erasure operations produce attributable records.

**FR-27 Data export**

Source or stakeholder need: API and storage path.

Proposed acceptance evidence: An authorized customer can export a bounded time range with schema version and integrity metadata.

**FR-28 Erasure request**

Source or stakeholder need: Compliance API and data stores.

Proposed acceptance evidence: An approved erasure request identifies all primary, derived, cached and backup copies and records completion evidence.

**FR-29 Health reporting**

Source or stakeholder need: Health and metrics routes.

Proposed acceptance evidence: Health separates liveness, readiness, dependency state and degraded-but-running modes.

**FR-30 Agent update verification**

Source or stakeholder need: Release and signing paths.

Proposed acceptance evidence: An update is authenticated, version-pinned, rollback-capable and rejected when signature or compatibility validation fails.

**FR-31 Windows capability declaration**

Source or stakeholder need: Windows agent path.

Proposed acceptance evidence: A release declares supported event classes and tested Windows versions; unsupported parity is visible to operators.

**FR-32 Configuration validation**

Source or stakeholder need: Compose, services and agent settings.

Proposed acceptance evidence: Startup fails closed on missing required configuration and reports safe, actionable validation errors.

**NFR-01 Least privilege**

Source or stakeholder need: Host and container deployment.

Proposed acceptance evidence: Privileges are reduced to the minimum needed for declared hooks and service tasks; exceptions have a threat rationale.

**NFR-02 Confidentiality**

Source or stakeholder need: Telemetry transport and storage.

Proposed acceptance evidence: Sensitive fields are encrypted in transit and at rest according to the deployment threat model.

**NFR-03 Availability**

Source or stakeholder need: Pipeline and services.

Proposed acceptance evidence: A dependency outage is visible and bounded; the endpoint's resource use and buffer limit remain within an approved policy.

**NFR-04 Recoverability**

Source or stakeholder need: WAL, Parquet and backups.

Proposed acceptance evidence: A restore drill validates data integrity, point-in-time scope and operator steps using a disposable copy.

**NFR-05 Privacy minimisation**

Source or stakeholder need: Event schema and dashboard.

Proposed acceptance evidence: Each field has a purpose, access rule and retention decision; command arguments and paths can be limited or redacted.

**NFR-06 Auditability**

Source or stakeholder need: Rules, models and actions.

Proposed acceptance evidence: Changes are versioned and attributable; inference and response records include the relevant model/rule version.

**NFR-07 Portability**

Source or stakeholder need: Linux build and Compose.

Proposed acceptance evidence: The project publishes and tests a supported OS/kernel/runtime matrix with reproducible dependency versions.

**NFR-08 Usability**

Source or stakeholder need: Operator dashboard.

Proposed acceptance evidence: A new operator can explain an alert and locate its evidence using a task-based usability evaluation.

**NFR-09 Accessibility**

Source or stakeholder need: Web frontend.

Proposed acceptance evidence: Core dashboard flows support keyboard navigation, meaningful labels and visible status text.

**NFR-10 Performance**

Source or stakeholder need: Agent and server.

Proposed acceptance evidence: Resource and event-latency targets are agreed before load testing and reported with hardware and workload metadata.

**NFR-11 Maintainability**

Source or stakeholder need: Service boundaries.

Proposed acceptance evidence: Public interfaces have versioned schemas and component owners; drift between README, container and runtime versions is removed.

**NFR-12 Safety**

Source or stakeholder need: Autonomous response.

Proposed acceptance evidence: Default mode is observe-only; high-impact actions require explicit opt-in, authenticated review and tested recovery.

### C.2 Selected verification protocols

The following tests are a proposed controlled verification plan derived from source interfaces and known gaps. No procedure in this appendix was executed for this report. Tests involving kernel hooks, response actions, deception, destructive file operations, or host-level simulations must run in isolated disposable systems with explicit authorization and a tested recovery plan. Capture full environment metadata and raw outputs before interpreting a pass or failure.

**G-01 Event schema round-trip**

Objective: Confirm the producer, Python bridge and storage layer agree on required fields.

Preconditions: Build a synthetic event fixture and the versioned schema validators.

Procedure: Serialize in the C/Python boundary format, ingest it, query it back, and compare types and values.

Expected result: All declared fields round-trip; unsupported event types are counted and safely handled.

Evidence to retain: Fixture, schema version, raw payload, query output and comparison log.

**G-03 Malformed JSON input**

Objective: Check the TCP listener's response to invalid records.

Preconditions: Isolated ingestion service and a client sending invalid, truncated and oversized lines.

Procedure: Submit each input class and continue sending a valid event afterward.

Expected result: Bad records are rejected or counted; the listener remains available for the valid record.

Evidence to retain: Client transcript, response codes, logs and queue state.

**G-04 Event batching boundary**

Objective: Verify batches flush at size and time boundaries.

Preconditions: Bridge configured with a test server and controllable clock or short test timeout.

Procedure: Send one event, then exactly a batch, then a partial batch while tracking writes.

Expected result: Each accepted record is sent once within the documented flush bound.

Evidence to retain: Sequence identifiers, packet capture in the lab, flush log and counts.

**G-05 Backend outage and retry**

Objective: Establish whether records are dropped, retried or buffered when the pipeline is unavailable.

Preconditions: Disposable endpoint and pipeline; outage injection isolated from production.

Procedure: Stop the receiver, generate a bounded event set, restore it and compare sequence IDs.

Expected result: Loss, duplicates and retry exhaustion are explicitly reported and remain within an agreed limit.

Evidence to retain: Before/after IDs, queue depth, retry counters and timing.

**G-06 Agent CPU and memory overhead**

Objective: Measure endpoint cost under controlled idle and event workloads.

Preconditions: Declared Linux kernel, host baseline, agent build and representative synthetic workload.

Procedure: Record baseline, run idle agent, then step through increasing event rates with repeated trials.

Expected result: Measurements are reproducible and compared with project-approved thresholds; no threshold is asserted here.

Evidence to retain: Host metadata, raw time series, load profile and analysis notebook.

**G-07 Supported kernel build matrix**

Objective: Test BPF compilation, load and detach across declared kernel versions.

Preconditions: Clean disposable VMs matching the proposed support matrix.

Procedure: Build from the pinned commit, load each program, generate one safe event and detach.

Expected result: Each supported combination has captured build and attach results; unsupported cases fail clearly.

Evidence to retain: Kernel/BTF details, compiler versions, build logs and event proof.

**G-09 Ring-buffer pressure**

Objective: Observe behaviour when the event consumer is slower than the producer.

Preconditions: Lab host, bounded load generator and enabled loss accounting.

Procedure: Increase event volume and introduce controlled reader delay.

Expected result: Drops, queue pressure and recovery are visible; the host remains stable.

Evidence to retain: Load profile, BPF counters, process resource logs and host health.

**G-10 Rule positive and negative fixtures**

Objective: Check each deterministic rule for intended triggers and benign near-misses.

Preconditions: Rule inventory, synthetic events and a reviewed benign command corpus.

Procedure: Run one positive and multiple boundary/negative fixtures per rule.

Expected result: Expected rules trigger with stable IDs and reasons; benign variants do not trigger unexpectedly.

Evidence to retain: Fixture set, per-rule output and reviewer adjudication.

**G-12 Window boundary and expiry**

Objective: Verify aggregation at the edges of the configured time window.

Preconditions: Clock-controlled events and isolated aggregator instance.

Procedure: Place events just before, at and after the window boundary; test idle expiry.

Expected result: Window membership and reset behaviour match the stated design.

Evidence to retain: Input timestamps, configuration, aggregate state and output.

**G-14 EWMA baseline update**

Objective: Validate baseline updates against hand-calculated expected values.

Preconditions: Small controlled sequence and a baseline reset mechanism.

Procedure: Apply deterministic observations and compare each state update with the specified alpha.

Expected result: Updates and entity/population weighting match the declared mathematics.

Evidence to retain: Input sequence, parameter configuration and computed trace.

**G-15 Baseline poisoning resistance**

Objective: Determine whether anomalous events can make the baseline permissive.

Preconditions: Isolated model store, synthetic normal sequence and labelled anomaly sequence.

Procedure: Compare normal updates, injected anomalies, freeze and rollback behaviour.

Expected result: Contamination impact is measured and an operator can restore a known-good baseline.

Evidence to retain: Baseline snapshots, event labels, threshold trajectory and recovery proof.

**G-16 Isolation Forest model provenance**

Objective: Establish the model input, version and training provenance.

Preconditions: Pinned model artefact and metadata with a reproducible training environment.

Procedure: Load, score a fixed vector set and compare output with the recorded model version.

Expected result: Provenance and deterministic scoring are auditable; no quality conclusion is made without labelled evaluation.

Evidence to retain: Hashes, training configuration, metadata and fixed-input outputs.

**G-17 Threshold sample minimum**

Objective: Check fallback behaviour before the calibration minimum is reached.

Preconditions: AdaptiveThresholds with a cleared store and a controlled pillar key.

Procedure: Submit fewer observations than the minimum, then the minimum and one more.

Expected result: Fallback, sample count and transition to calibrated output are correct and visible.

Evidence to retain: Stored counts, thresholds and service logs.

**G-19 Threshold rollback**

Objective: Confirm calibration state can return to an approved snapshot.

Preconditions: Disposable database and saved baseline snapshot.

Procedure: Change thresholds, restore the snapshot and compare all keys and metadata.

Expected result: Restored values match the snapshot; restoration is attributed and logged.

Evidence to retain: Before/after export, restore log and hash comparison.

**G-20 Governor risk downgrade**

Objective: Check high criticality or blast radius reduces a proposed action tier.

Preconditions: Unit-level governor fixture with safe in-memory inputs.

Procedure: Test values below, at and above each configured boundary.

Expected result: Decision matches code policy and includes a reason; no endpoint action is invoked.

Evidence to retain: Input matrix, decision output and expected policy mapping.

**G-21 Human confirmation flow**

Objective: Verify confirmation requirements and timeout behaviour.

Preconditions: Mock identity service and non-executing response adapter.

Procedure: Request a guarded action, approve as authorized user, deny, and allow a request to expire.

Expected result: Only the valid explicit approval produces an auditable decision; timeout defaults to no action.

Evidence to retain: Request IDs, user identity claims, timestamps, audit events and adapter logs.

**G-22 Tenant isolation read paths**

Objective: Test customer separation for every API list and detail endpoint.

Preconditions: Two isolated synthetic tenants, test users and known event IDs.

Procedure: Attempt permitted and cross-tenant reads using each API route and filter combination.

Expected result: No unauthorized record is returned; denial is consistent and logged.

Evidence to retain: Route matrix, request/response samples and audit log.

**G-23 Tenant isolation mutation paths**

Objective: Test updates, actions and bulk routes against cross-customer targets.

Preconditions: Two synthetic tenants and a mocked no-op action handler.

Procedure: Attempt cross-tenant mutations with valid and invalid identities.

Expected result: Unauthorized actions are denied before side effects; attribution is retained.

Evidence to retain: Request set, mocked handler calls and audit records.

**G-24 Credential rotation and secret scan**

Objective: Confirm runtime secrets are externalized and rotatable.

Preconditions: Disposable deployment with test-only credentials and secret scanning tools.

Procedure: Review repository/config, rotate a credential and restart dependent services.

Expected result: No active secret is committed; revoked credentials fail and the rotated value works only in the authorized store.

Evidence to retain: Secret-scan report with redactions, rotation timeline and restart evidence.

**G-27 Pipeline restart recovery**

Objective: Measure durable event recovery after a controlled process restart.

Preconditions: Disposable storage volume and known sequence of events.

Procedure: Record events, stop process at planned points, restart and compare persisted IDs.

Expected result: Recovery behaviour matches documented acknowledgement and durability semantics.

Evidence to retain: Event sequence, process logs, database query and loss/duplicate report.

**G-29 Retention and erasure verification**

Objective: Trace deletion across queryable stores and backups.

Preconditions: Synthetic tenant data with short test retention and disposable backup copy.

Procedure: Expire and erase selected records, then search primary, derived and restored data.

Expected result: All in-scope copies are accounted for and deletion completion is auditable.

Evidence to retain: Record IDs, store queries, backup test and erasure audit entry.

**G-30 Dashboard failure states**

Objective: Assess loading, empty, forbidden, timeout and stale-data states.

Preconditions: Frontend test harness with controlled API responses.

Procedure: Exercise each state, keyboard navigation and retry flow.

Expected result: The operator can distinguish no alerts from an API failure and stale information is labelled.

Evidence to retain: Automated UI output, accessibility scan and review checklist.

**G-33 API authorization and injection**

Objective: Check route authorization, input validation and query safety.

Preconditions: Staging API, fixture database and approved security test cases.

Procedure: Exercise unauthenticated routes, invalid identifiers, boundary strings and injection payloads.

Expected result: Sensitive routes require identity; inputs are validated and query data remains isolated.

Evidence to retain: Route matrix, redacted traffic capture, logs and finding register.

**G-34 Dependency and configuration reproducibility**

Objective: Check clean deployment from pinned files and documented versions.

Preconditions: Fresh disposable host with no cached images or local secrets.

Procedure: Build from pinned dependencies and compare configured runtime versions.

Expected result: Build is repeatable and all version discrepancies are resolved or explicitly supported.

Evidence to retain: Dependency lock state, image digests, build logs and configuration inventory.

**G-35 Soak and queue stability**

Objective: Measure long-duration buffer, stream and storage behaviour.

Preconditions: Isolated system with a pre-agreed workload and safe disk budget.

Procedure: Run stepped, sustained synthetic event traffic and periodically sample all queue/store metrics.

Expected result: No unbounded memory growth or silent loss occurs within the approved workload envelope.

Evidence to retain: Resource timeseries, event reconciliation, queue lag and service logs.

**G-36 False-positive adjudication**

Objective: Estimate alerts on benign SME-like workflows.

Preconditions: Documented, consented, representative benign workload and an adjudication rubric.

Procedure: Run detectors in observe-only mode, label each alert independently and review disagreements.

Expected result: Report false-positive rate with workload composition and confidence interval; do not extrapolate beyond the sample.

Evidence to retain: Dataset description, adjudication records, script versions and statistical summary.

### C.3 SME pilot and evaluation plan

This schedule is a proposal for a future, consented evaluation. It is not a pilot already performed. Begin with synthetic data and isolated systems, then move to a small number of approved endpoints only after installation, privacy, rollback and operator responsibilities have been reviewed. Keep the system in observe-only mode throughout the initial pilot.

**Table C.1 Proposed staged pilot timeline**

| Stage | Period | Activities and exit evidence |
|---|---|---|
| Governance and scope | Weeks 1–2 | Agree participants, systems in scope, data minimisation, consent, retention, incident ownership and rollback authority. Record the approved workload and safety boundaries. |
| Reproducible build | Weeks 3–4 | Resolve documented runtime-version differences, pin dependencies, externalise secrets and build from a clean environment. Retain hashes, build logs and host compatibility details. |
| Synthetic verification | Weeks 5–6 | Exercise event mapping, rules, storage, restart recovery, API authorization and dashboard failure states with synthetic events. Save raw inputs, logs and expected-versus-actual results. |
| Isolated endpoint trial | Weeks 7–8 | Install only on disposable or explicitly approved Linux hosts. Measure event coverage, loss indicators and resource impact in observe-only mode; make no automated host changes. |
| Workload evaluation | Weeks 9–10 | Collect a consented benign workload and run controlled simulations only in an isolated lab. Adjudicate alerts independently and freeze model/threshold versions during each comparison. |
| Recovery and review | Weeks 11–12 | Verify backup, restoration, data expiry and deletion; review false alerts, latency, overhead and operator feedback. Issue a go/no-go decision with unresolved risks and evidence attached. |

A pilot should stop if event integrity cannot be measured, customer boundaries are unclear, credentials remain embedded in source/configuration, data handling is not approved, or a response action could affect a production endpoint without explicit human authorization.

**Table C.2 Measures and reporting boundaries**

| Measure | Evidence to collect | Reporting boundary |
|---|---|---|

| Measure | Evidence to collect | Reporting boundary |
|---|---|---|
| Event coverage and integrity | Compare generated fixture identifiers with received, stored and queried identifiers; record drops, duplicates and unknown event types. | Report the exact host, kernel, event set and observation window. Do not call a source-declared hook complete without observed evidence. |
| Alert quality | Use labelled benign and controlled test cases, independent adjudication and a documented confusion matrix. | Report precision/recall with dataset composition and uncertainty. Do not extrapolate a small pilot to other SMEs. |
| Processing latency | Timestamp collection, receipt, detection and display; summarize p50 and p95 under a stated workload. | State clock method, volume, host resources and test duration. No value is currently established by this report. |
| Endpoint and service overhead | Measure CPU, memory, disk growth, queue depth and network use against a comparable baseline. | Report configuration and sampling method; distinguish agent overhead from other services. |
| Reliability and recovery | Reconcile event IDs before and after restart, storage pressure, backup and restoration procedures. | Disclose loss, duplication and recovery time; repository code alone does not prove durability. |
| Privacy and safety | Review captured fields, access logs, retention/deletion evidence, approvals and rollback records. | Identify sensitive fields and who could access them. Keep high-impact response disabled until separately validated. |

### C.4 Evidence-status key

The following labels are used consistently in the report so that repository code, proposed work and measured outcomes are not confused.

**Table C.3 Evidence-status key**

| Status label | Meaning in this report |
|---|---|
| Implemented in source | Relevant code or configuration exists in the reviewed commit; runtime behaviour is not implied. |
| Partial | Only part of the intended capability is represented, or repository documentation identifies unfinished work. |
| Planned or explicit gap | A desired capability, input, control or validation step is recorded as missing or deferred. |
| Unverified | Files or routes exist, but integration, security, quality or operational evidence is insufficient. |
| Not executed | A test artefact or proposed protocol was inspected but was not run while preparing this report. |
| Measured result | Use only when raw output, environment details, method and reproducible analysis are retained. |

## REFERENCES


[1] Ghost Layer Technologies, Ghost IT repository, branch main, commit 4dfb877c5aaad23e456fa4bbd0c0bdba761b2fdb, 25 August 2026. Accessed 22 October 2026. https://github.com/K-v234/ghostlayer-ghostit

[2] C. Pascoe, S. Quinn and K. Scarfone, The NIST Cybersecurity Framework (CSF) 2.0, NIST Cybersecurity White Paper 29, National Institute of Standards and Technology, 2024. https://doi.org/10.6028/NIST.CSWP.29

[3] Linux Kernel Organization, eBPF Userspace API, Linux Kernel Documentation. Accessed 22 October 2026. https://kernel.org/doc/html/latest/userspace-api/ebpf/index.html

[4] F. T. Liu, K. M. Ting and Z.-H. Zhou, Isolation Forest, in Proceedings of the 8th IEEE International Conference on Data Mining, 2008, pp. 413-422. https://doi.org/10.1109/ICDM.2008.17

[5] DuckDB Foundation, DuckDB Documentation. Accessed 22 October 2026. https://duckdb.org/docs/stable/

[6] Redis Ltd., Redis Streams Documentation. Accessed 22 October 2026. https://redis.io/docs/latest/develop/data-types/streams/

[7] MITRE, MITRE ATT&CK Enterprise knowledge base. Accessed 22 October 2026. https://attack.mitre.org/